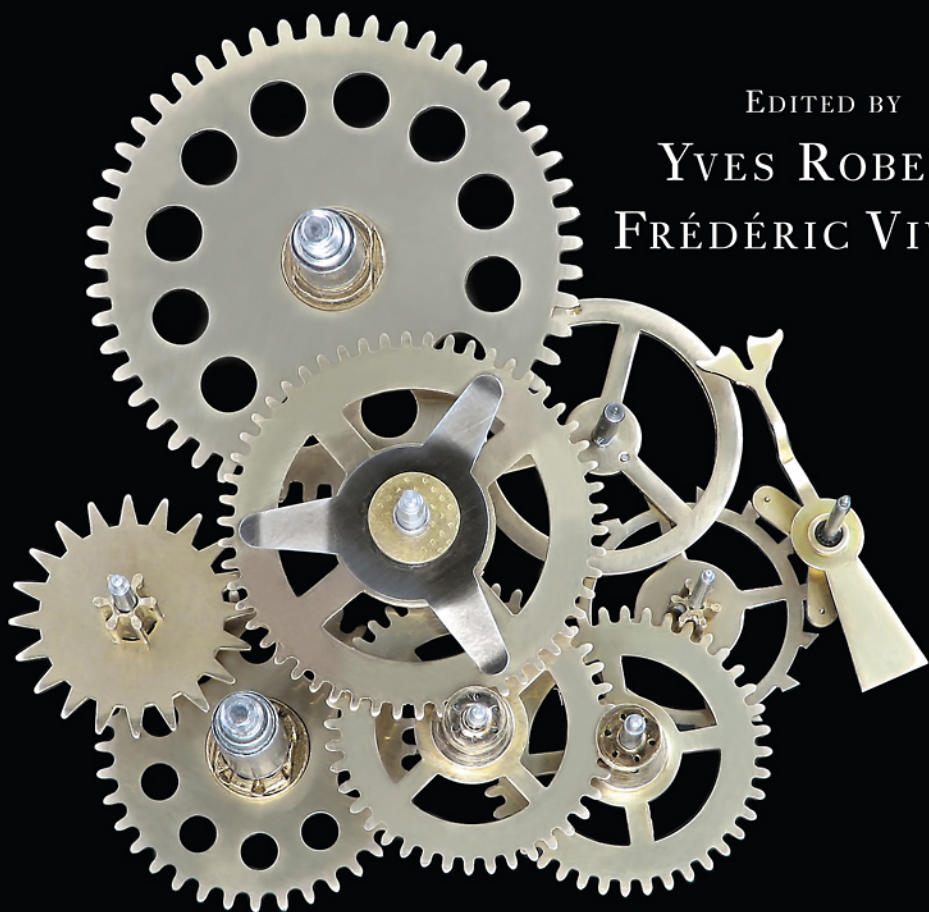


Chapman & Hall/CRC
Computational Science Series

INTRODUCTION TO Scheduling

EDITED BY
YVES ROBERT
FRÉDÉRIC VIVIEN



CRC Press
Taylor & Francis Group

A CHAPMAN & HALL BOOK

INTRODUCTION TO

Scheduling

Chapman & Hall/CRC

Computational Science Series

SERIES EDITOR

Horst Simon

Associate Laboratory Director, Computing Sciences

Lawrence Berkeley National Laboratory

Berkeley, California, U.S.A.

AIMS AND SCOPE

This series aims to capture new developments and applications in the field of computational science through the publication of a broad range of textbooks, reference works, and handbooks. Books in this series will provide introductory as well as advanced material on mathematical, statistical, and computational methods and techniques, and will present researchers with the latest theories and experimentation. The scope of the series includes, but is not limited to, titles in the areas of scientific computing, parallel and distributed computing, high performance computing, grid computing, cluster computing, heterogeneous computing, quantum computing, and their applications in scientific disciplines such as astrophysics, aeronautics, biology, chemistry, climate modeling, combustion, cosmology, earthquake prediction, imaging, materials, neuroscience, oil exploration, and weather forecasting.

PUBLISHED TITLES

PETASCALE COMPUTING: Algorithms and Applications

Edited by David A. Bader

PROCESS ALGEBRA FOR PARALLEL AND DISTRIBUTED PROCESSING

Edited by Michael Alexander and William Gardner

GRID COMPUTING: TECHNIQUES AND APPLICATIONS

Barry Wilkinson

INTRODUCTION TO CONCURRENCY IN PROGRAMMING LANGUAGES

Matthew J. Sottile, Timothy G. Mattson, and Craig E Rasmussen

INTRODUCTION TO SCHEDULING

Yves Robert and Frédéric Vivien

INTRODUCTION TO
Scheduling

EDITED BY
YVES ROBERT
FRÉDÉRIC VIVIEN



CRC Press

Taylor & Francis Group
Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group an **informa** business
A CHAPMAN & HALL BOOK

CRC Press
Taylor & Francis Group
6000 Broken Sound Parkway NW, Suite 300
Boca Raton, FL 33487-2742

© 2010 by Taylor and Francis Group, LLC
CRC Press is an imprint of Taylor & Francis Group, an Informa business

No claim to original U.S. Government works

Printed in the United States of America on acid-free paper
10 9 8 7 6 5 4 3 2 1

International Standard Book Number: 978-1-4200-7273-0 (Hardback)

This book contains information obtained from authentic and highly regarded sources. Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, please access www.copyright.com (<http://www.copyright.com/>) or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. CCC is a not-for-profit organization that provides licenses and registration for a variety of users. For organizations that have been granted a photocopy license by the CCC, a separate system of payment has been arranged.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

Library of Congress Cataloging-in-Publication Data

Introduction to scheduling / Yves Robert, Frederic Vivien.

p. cm. -- (Chapman & Hall/CRC computational science series)

Includes bibliographical references and index.

ISBN 978-1-4200-7273-0 (hardcover : alk. paper)

1. Electronic data processing--Distributed processing. 2. Computer scheduling--Mathematical models. 3. Multiprocessors. 4. Computational grids (Computer systems) I. Robert, Yves, 1938- II. Vivien, Frédéric. III. Title. IV. Series.

QA76.9.D51673 2009
004'.36--dc22

2009032786

Visit the Taylor & Francis Web site at
<http://www.taylorandfrancis.com>

and the CRC Press Web site at
<http://www.crcpress.com>

Contents

Preface	xi
Acknowledgments	xvii
List of Contributors	xix
1 On the Complexity of Scheduling	1
<i>Peter Brucker and Sigrid Knust</i>	
1.1 Introduction	1
1.2 Scheduling Models	2
1.3 Processor (Machine) Scheduling	6
1.4 Easy and Hard Problems	11
1.5 Complexity Classification of Scheduling Problems	18
References	20
2 Approximation Algorithms for Scheduling Problems	23
<i>Jean-Claude König and Rodolphe Giroudeau</i>	
2.1 Introduction	23
2.1.1 Approximation Algorithms	24
2.1.2 Definitions	25
2.1.3 Absolute Approximation Algorithms (Case $\rho_{inf} = \rho_{sup}$)	26
2.2 A Fully Polynomial-Time Approximation Scheme	28
2.3 Introduction of Precedence Constraints	31
2.3.1 Unbounded Number of Processors	31
2.3.2 Bounded Number of Processors	31
2.4 Introduction of Communication Delays	32
2.4.1 Introduction	32
2.4.2 Unbounded Number of Processors	33
2.4.3 Limited Number of Processors	37
2.4.4 Introduction of Duplication	42
2.4.5 Large Communication Delays	44
2.5 Conclusion	47
References	48
3 Online Scheduling	51
<i>Susanne Albers</i>	
3.1 Introduction	51
3.2 Classical Scheduling Problems	52
3.2.1 Makespan Minimization	52
3.2.2 Flow Time Objectives	57

3.2.3	Load Balancing	60
3.3	Energy-Efficient Scheduling	62
3.3.1	Power-Down Mechanisms	63
3.3.2	Dynamic Speed Scaling	67
3.4	Conclusion	73
	References	73
4	Job Scheduling	79
	<i>Uwe Schwiegelshohn</i>	
4.1	Introduction	79
4.2	Single Machine Problems	82
4.3	Makespan Problems on Parallel Machines	86
4.4	Completion Time Problems on Parallel Machines	91
4.5	Conclusion	99
	References	100
5	Cyclic Scheduling	103
	<i>Claire Hanen</i>	
5.1	Introduction	103
5.2	Cyclic Scheduling and Uniform Constraints	104
5.2.1	Common Features of Cyclic Scheduling Problems	104
5.2.2	Uniform Task Systems	106
5.2.3	Questions	108
5.3	Periodic Schedules of Uniform Task Systems	109
5.3.1	Properties of Periodic Schedules	109
5.3.2	Critical Circuit of a Strongly Connected Graph	112
5.3.3	Computation of an Optimal Periodic Schedule	113
5.4	Earliest Schedule of Uniform Task Systems	116
5.5	Periodic Schedules of Uniform Task Systems with Resource Constraints	117
5.5.1	Which Periodicity?	117
5.5.2	Complexity and MIP Models	118
5.5.3	Patterns and Iteration Vectors	119
5.5.4	Decomposed Software Pipelining: A Generic Approach	121
5.6	Dynamic Schedules	124
5.7	Conclusion	125
	References	126
6	Cyclic Scheduling for the Synthesis of Embedded Systems	129
	<i>Olivier Marchetti and Alix Munier-Kordon</i>	
6.1	Introduction	129
6.2	Problem Formulation and Basic Notations	131
6.2.1	Synchronous Dataflow Graphs	131
6.2.2	Timed Weighted Event Graphs	132
6.2.3	Problem Formulation	133

6.3	Precedence Relations Induced by a Timed Marked WEG . . .	134
6.3.1	Characterization of Precedence Relations	134
6.3.2	Timed Event Graphs	135
6.3.3	Equivalent Places	135
6.4	Unitary WEGs	137
6.4.1	Definitions	138
6.4.2	Normalization of a Unitary WEG	139
6.4.3	Expansion of a Unitary Timed Marked WEG	141
6.4.4	Relationship between Expansion and Normalization	145
6.5	Periodic Schedule of a Normalized Timed Marked WEG . . .	147
6.5.1	Periodic Schedules	148
6.5.2	Properties of Periodic Schedules	148
6.5.3	Existence of Periodic Schedules	150
6.5.4	Optimal Periodic Schedule	152
6.6	Conclusion	154
	References	154
7	Steady-State Scheduling	159
	<i>Olivier Beaumont and Loris Marchal</i>	
7.1	Introduction	159
7.2	Problem Formulation	161
7.2.1	Platform Model	161
7.2.2	Applications	162
7.3	Compact Description of a Schedule	163
7.3.1	Definition of the Allocations	164
7.3.2	Definition of Valid Patterns	166
7.4	From Allocations and Valid Patterns to Schedules	167
7.4.1	Conditions and Weak Periodic Schedules	167
7.4.2	Weak Periodic Schedules and Cyclic Scheduling	169
7.5	Problem Solving in the General Case	172
7.5.1	Existence of a Compact Solution	173
7.5.2	Resolution with the Ellipsoid Method	175
7.5.3	Separation in the Dual Linear Program	176
7.6	Toward Compact Linear Programs	178
7.6.1	Introduction	178
7.6.2	Efficient Computation of Valid Patterns under the Bidi- rectional One-Port Model	179
7.6.3	Efficient Computation of Allocations	182
7.7	Conclusion	184
	References	185

8 Divisible Load Scheduling 187

Matthieu Gallet, Yves Robert, and Frédéric Vivien

8.1	Introduction	187
8.1.1	Motivating Example	188
8.1.2	Classical Approach	188
8.2	Divisible Load Approach	191
8.2.1	Bus-Shaped Network	192
8.2.2	Star-Shaped Network	195
8.3	Extensions of the Divisible Load Model	201
8.3.1	Introducing Latencies	201
8.3.2	Multi-Round Strategies	204
8.3.3	Return Messages	214
8.4	Conclusion	216
	References	217

9 Multi-Objective Scheduling 219

Pierre-François Dutot, Krzysztof Rzadca, Erik Saule, and Denis Trystram

9.1	Motivation	220
9.1.1	Once Upon a Time	220
9.1.2	Diversity of Objectives	221
9.1.3	Motivating Problems	222
9.1.4	Summary of Results on Single Objective Problems	223
9.1.5	Beyond the Scope of This Chapter	223
9.1.6	Chapter Organization	224
9.2	What Is Multi-Objective Optimization?	225
9.3	Overview of the Various Existing Approaches	228
9.3.1	Algorithms Building One Trade-off Solution	228
9.3.2	Complexity Issues	230
9.4	Zenith Approximation on MAXANDSUM	233
9.5	Pareto Set Approximation on EFFICIENTRELIABLE	235
9.5.1	Motivation	235
9.5.2	Definition of Pareto Set Approximation	236
9.5.3	The Thresholding Approach	237
9.6	Fairness as Multi-Objective Optimization	241
9.6.1	The Meaning of Fairness	241
9.6.2	Axiomatic Theory of Fairness	242
9.6.3	Application to TWOAGENTMINSUM	243
9.6.4	Problems with Different Objective Functions	245
9.6.5	Aggregative Fairness	246
9.7	Conclusion	247
	References	248

10 Comparisons of Stochastic Task-Resource Systems 253

Bruno Gaujal and Jean-Marc Vincent

10.1 Motivation	253
10.2 Task-Resource Models	255
10.2.1 Static Systems	255
10.2.2 Dynamic Systems	256
10.3 Stochastic Orders	257
10.3.1 Orders for Real Random Variables	258
10.3.2 Orders for Multidimensional Random Variables	263
10.3.3 Association	264
10.4 Applications to Static Problems	265
10.4.1 The $1 \sum C_i$ Problem, Revisited	266
10.4.2 PERT Graphs	267
10.5 Applications to Dynamic Systems	268
10.5.1 Single Queues	269
10.5.2 Networks of Queues	272
10.5.3 Stochastic Comparisons and Simulation Issues	275
References	279

11 The Influence of Platform Models on Scheduling Techniques 281

Lionel Eyraud-Dubois and Arnaud Legrand

11.1 Introduction	281
11.2 Platform Modeling	282
11.2.1 Modeling the Topology	282
11.2.2 Modeling Point-to-Point Communication Time	284
11.2.3 Heterogeneity	288
11.2.4 Modeling Concurrent Communications	289
11.2.5 Interaction between Communication and Computation	291
11.3 Scheduling Divisible Load	292
11.3.1 Single Round	293
11.3.2 Multi-Round	296
11.4 Iterative Algorithms on a Virtual Ring	298
11.4.1 Problem Statement	299
11.4.2 Complete Homogeneous Platform	299
11.4.3 Complete Heterogeneous Platform	300
11.4.4 Arbitrary Heterogeneous Platform	301
11.5 Data Redistribution	302
11.5.1 The Matching Approach	304
11.5.2 The Elastic Flows Approach	306
11.6 Conclusion	307
References	307

Preface

Aim and Scope

The objective of this book is twofold:

- To introduce the basic concepts, methods, and fundamental results of scheduling theory.
- To expose recent developments in the area.

In each chapter, emphasis is given to self-contained, easy-to-follow, but still rigorous presentations. The book is full of examples, beautiful theorems, and pedagogical proofs. It also contains an in-depth coverage of some key application fields. Altogether, this book provides a comprehensive overview of important results in the field, organized as a handy reference for students and researchers.

There are numerous books and proceedings devoted to scheduling techniques, among others [1, 2, 3, 4]. However, we believe that this book is unique, because it is meant to be accessible to a large audience while covering both the foundations and modern developments.

Book Overview

For the sake of presentation, we have organized the chapters into five main categories: *Foundations*, *Online and Job Scheduling*, *Cyclic Scheduling*, *Advanced Topics*, and *Platform Models*. However, each chapter can be read independently, or following the pointers to relevant material in other chapters whenever needed.

Foundations

The first two chapters present fundamental concepts. The purpose of Chapter 1 is to introduce and classify scheduling problems and their complexity. Furthermore, the notations used in the book are introduced. Firstly, the resource constrained project scheduling problem is defined and several applications of this problem are discussed. An important subclass is the class of machine scheduling problems. For a classification of these problems, the well-known three field notation is introduced. Secondly, the basic definitions of the theory of computational complexity are given and applied to machine scheduling problems. It is shown how the three field notation can be used

to identify, for different classes of machine scheduling problems, the hardest problems known to be polynomially solvable, the easiest NP-hard problems, and the minimal and maximal open problems.

Because many scheduling problems are NP-hard (hence it is quite unlikely that we will ever be able to find efficient polynomial-time algorithms for their solution), it is worth looking for algorithms that always return a feasible solution whose measure is not too far from the optimum. Chapter 2 is devoted to the study of such *approximation algorithms*. The fundamentals of this approach are first recorded via a classification of optimization problems according to their polynomial-time approximability. Some examples are then presented in order to show successful techniques for the design of efficient approximation algorithms, and some basic ideas of methods for proving lower bounds on polynomial-time inapproximability are provided.

Online and Job Scheduling

The next two chapters introduce the reader to two important related topics, *online* scheduling and *job* scheduling. Traditional scheduling theory assumes that an algorithm, when computing a schedule, has complete knowledge of the entire input. However, this assumption is often unrealistic in practical applications. In an online setting, jobs arrive over time; whenever a new job arrives scheduling decisions must be made without knowledge of any future jobs. On arrival of a job, its complete characteristics such as the processing time may or may not be known in advance. We seek scheduling algorithms that work well in online scenarios and have a proven good performance. In this context, an online algorithm A is called c -competitive if, for all inputs, the objective function value of a schedule computed by A is at most a factor of c away from that of an optimal schedule. In Chapter 3, we study a number of classical problems as well as fresh problems of current interest. As for classical problems, we consider, for instance, the famous makespan minimization problem in which jobs have to be scheduled on parallel machines so as to minimize the makespan of the resulting schedule. We analyze Graham's List scheduling algorithm, that is 2-competitive, as well as more recent algorithms that improve upon the factor of 2. With respect to recent advances, we study energy-efficient scheduling algorithms. Here jobs must be scheduled on a variable speed processor so as to optimize certain objective functions.

In Chapter 4, we target job scheduling problems that comprise independent and possibly parallel jobs. Total (weighted) completion or response times and machine utilization are the most common objective functions, although complex objectives defined by system owners have recently received increasing interest. The machine models are usually restricted to parallel identical machines but may consider communication delays and machine subsets. The schedules may either be preemptive or non-preemptive. Occasionally, some other constraints, like migration with penalties, are considered as well. Since basically all interesting deterministic job scheduling problems are NP-hard,

many theoretical results yield approximation factors. As in practice most job scheduling problems include unknown processing times and submission over time many online results are also given in the literature. However, in general, only simple algorithms, like various forms of list scheduling, are analyzed as complex approaches with potentially good competitive or approximation factors are of limited practical interest. In this chapter, we present several important results and explain how they are related to each other. Furthermore, we describe various methods to prove competitive or approximation factors of simple job scheduling algorithms.

Cyclic Scheduling

Cyclic scheduling problems occur when a set of tasks is to be repeated many times. Chapter 5 begins with a short introduction to applications which motivated the study of cyclic scheduling problems. Then we introduce the basic model for precedence relations between tasks in this context, called uniform precedences. Two main kinds of studies have been devoted to cyclic scheduling problems in this context. The first one aims at analyzing the behavior of the earliest schedule without any resource constraints. The second approach aims at defining efficient periodic schedules in contexts where resources are to be taken into account. We discuss the notion of periodicity, the complexity of these problems, and exhibit some special cases which can be polynomially solved. In the general case, we present an approach taken from the field of *software pipelining*. We also give a list of resource contexts in which such problems have been studied (parallel or dedicated processors, registers) with different optimization criteria.

The major objective of Chapter 6 is to present a practical application of the results of Chapter 5 for the synthesis of embedded systems. The first part of this chapter is devoted to the formulation of the practical problem using Generalized Timed Event Graphs (in short GEGT), a subclass of Petri Nets. The second part of this chapter is devoted to the presentation of some mathematical tools to study the properties of a GEGT. It has been shown that pseudo-polynomial algorithms can be developed to evaluate the liveness or the throughput of a GEGT. More recently, a nice simplification of the GEGT was introduced leading to a simple sufficient condition of liveness. Some new results on the evaluation of the throughput will also be presented.

Chapter 7 deals with *steady-state* scheduling, an approach that also builds upon cyclic scheduling techniques, but in the context of large scale platforms. Steady-state scheduling consists in optimizing the number of tasks that can be processed per time unit when the number of tasks becomes arbitrarily large. In this chapter, we concentrate on bag-of-tasks applications and collective communications (broadcast and multicast) and we prove that efficient schedules can be derived in the context of steady-state scheduling, under realistic communication models that take into account both the heterogeneity of the resources and the contentions in the communication network.

Advanced Topics

Chapter 8 focuses on scheduling large and compute-intensive applications on parallel resources, typically organized as a master-worker platform. We assume that we can arbitrarily split the total work, or load, into chunks of arbitrary sizes, and that we can distribute these chunks to an arbitrary number of workers. The job has to be perfectly parallel, without any dependence between sub-tasks. In practice, this model, known as the *Divisible Load* model, is a reasonable relaxation of an application made up of a large number of identical, fine-grain parallel computations; instead of scheduling an integer number of tasks on each resource, we allow for a rational number of tasks. The chapter provides several examples of divisible load computations, and discusses several extensions to assess both the power and the limitations of the divisible model.

Chapter 9 considers multi-objective scheduling, i.e., scheduling with the goal of simultaneously optimizing several objectives. In a multi-objective setting, one can explicitly model, optimize, and trade off between various performance measures. The chapter provides three motivating problems to illustrate different approaches. In *MaxAndSum*, a scheduler optimizes both the makespan and the sum of completion times. In *EfficientReliable*, the goal is to find the trade-off between the makespan and the reliability when scheduling on failing processors. Finally, in *TwoAgentMinSum*, a processor must be shared fairly between jobs produced by two independent users. In this latter example, we propose an axiomatic way to characterize schedules that are *fair* to individual users.

Chapter 10 shows how to compare the performance of stochastic task resource systems. Here, task resource systems are modeled by dynamic systems whose inputs are the task and resource data, given under the form of random processes, and whose outputs are the quantities of interest (response times, makespan). The approach presented here is based on stochastic orders. Several typical examples (some simple, others rather sophisticated) are used to show how orders can help the system designer to compare several options but also to compute bounds on the performance of a given system.

Platform Models

Chapter 11, the last chapter of the book, aims at assessing the impact of platform models on scheduling techniques. First, we survey a wide and representative panel of platform models and topologies used in the literature. Then we explain why such models have been used in the past and why they may be ill-adapted to current applications and current platforms. Next we present through a few simple examples the impact of these models on the complexity of the corresponding problems. Lastly we present a few promising network models for modern platforms, discussing their realism as well as their tractability.

References

- [1] J. Blazewicz, K. H. Ecker, E. Pesch, G. Schmidt, and J. Weglarz, editors. *Handbook on Scheduling: From Theory to Applications*. Springer, 2007.
- [2] P. Chrétienne, E. G. Coffman Jr., J. K. Lenstra, and Z. Liu, editors. *Scheduling Theory and its Applications*. John Wiley & Sons, 1995.
- [3] J. Y.-T. Leung, editor. *Handbook of Scheduling: Algorithms, Models, and Performance Analysis*. Chapman & Hall/CRC, 2004.
- [4] B. A. Shirazi, A. R. Hurson, and K. M. Kavi. *Scheduling and load balancing in parallel and distributed systems*. IEEE Computer Science Press, 1995.

Acknowledgments

This book originates from the invited talks that were given during the 35th French Spring School in Theoretical Computer Science (*EPIT'2007*), which took place in June 2007 at La Villa Clythia, Fréjus, in southern France. We thank *CNRS*—the French Council for National Research—and *INRIA*—the French National Institute for Research in Computer Science and Control—whose support made the school possible.

We also thank the whole editorial team at CRC Press, whose encouragement and friendly pressure helped us finalize this project (almost in time).

Finally, we sincerely thank all chapter authors for their contributions. It was really nice working with this great coterie of French and German colleagues!

Yves Robert and Frédéric Vivien
Lyon and Honolulu

Contact: {Yves.Robert|Frederic.Vivien}@ens-lyon.fr

Contributors

Susanne Albers

Humboldt University Berlin
Berlin, Germany

Olivier Beaumont

INRIA
Bordeaux, France

Peter Brucker

University of Osnabrück
Osnabrück, Germany

Pierre-François Dutot

Université de Grenoble
Grenoble, France

Lionel Eyraud-Dubois

INRIA
Université de Bordeaux
Bordeaux, France

Matthieu Gallet

École normale supérieure de Lyon
Université de Lyon
Lyon, France

Bruno Gaujal

INRIA
Université de Grenoble
Grenoble, France

Rodolphe Giroudeau

LIRMM
Montpellier, France

Claire Hanen

Université Paris Ouest
Nanterre-La Défense
Nanterre, France

Sigrid Knust

Institute of Mathematics
Technical University of Clausthal
Clausthal, Germany

Jean-Claude König

LIRMM
Montpellier, France

Arnaud Legrand

CNRS
Université de Grenoble
Grenoble, France

Loris Marchal

CNRS
Université de Lyon
Lyon, France

Olivier Marchetti

Université Pierre et Marie Curie
Paris 6
Paris, France

Alix Munier-Kordon

Université Pierre et Marie Curie
Paris 6
Paris, France

Yves Robert

École normale supérieure de Lyon
Institut Universitaire de France
Université de Lyon
Lyon, France

Krzysztof Rządca

Polish-Japanese Institute of
Information Technology
Warsaw, Poland

Erik Saule

Université de Grenoble
Grenoble, France

Uwe Schwiegelshohn

Technische Universität Dortmund
Dortmund, Germany

Denis Trystram

Université de Grenoble
Grenoble, France

Jean-Marc Vincent

INRIA
Université de Grenoble
Grenoble, France

Frédéric Vivien

INRIA
Université de Lyon
Lyon, France

Chapter 1

On the Complexity of Scheduling

Peter Brucker

University of Osnabrück

Sigrid Knust

Technical University of Clausthal

1.1	Introduction	1
1.2	Scheduling Models	2
1.3	Processor (Machine) Scheduling	6
1.4	Easy and Hard Problems	11
1.5	Complexity Classification of Scheduling Problems	18
	References	20

Abstract This chapter presents a survey on scheduling models (project scheduling and processor scheduling) and their computational complexity.

1.1 Introduction

Scheduling is concerned with the allocation of limited resources to activities over time. Activities may be tasks in computer environments, steps of a construction project, operations in a production process, lectures at the university, etc. Resources may be processors, workers, machines, lecturers, and so on. General scheduling models will be introduced and specific applications like project or processor scheduling will be discussed.

Methods for solving scheduling problems depend on the computational complexity. For machine (processor) scheduling problems a classification scheme has been introduced. After describing it we survey some polynomially solvable and NP-hard scheduling problems.

1.2 Scheduling Models

The so-called resource-constrained project scheduling problem (RCPSP, see also Brucker and Knust [9]) is one of the basic complex scheduling problems. In the following we introduce this problem and some of its generalizations. Processor (machine) scheduling problems, that may be considered as special cases, are introduced in Section 1.3.

The resource-constrained project scheduling problem (RCPSP) is a very general scheduling problem that may be used to model many applications in practice (e.g., a production process, a software project, a school timetable, the construction of a house, or the renovation of an airport). The objective is to schedule some activities over time such that scarce resource capacities are respected and a certain objective function is optimized. Examples for resources may be processors, machines, people, or rooms, which are only available with limited capacities. As objective functions the project duration, the deviation from deadlines, or costs concerning resources may be minimized.

The **resource-constrained project scheduling problem (RCPSP)** may be formulated as follows. Given are n **activities** (jobs) $j = 1, \dots, n$ and r **renewable resources** $k = 1, \dots, r$. A constant amount of R_k units of resource k is available at any time. Activity j must be processed for p_j time units. During this time period a constant amount of r_{jk} units of resource k is occupied. All data are assumed to be integers. A resource is called **disjunctive** if $R_k = 1$, otherwise it is called **cumulative**. If resource k is disjunctive, two activities i, j with $r_{ik} = r_{jk} = 1$ cannot be processed simultaneously.

Furthermore, **precedence constraints** are defined between some activities. They are given by relations $i \rightarrow j$, where $i \rightarrow j$ means that activity j cannot start before activity i is completed.

The objective is to determine starting times S_j for the activities $j = 1, \dots, n$ in such a way that

- at each time t the total resource demand is less than or equal to the resource availability of each resource $k = 1, \dots, r$,
- the given precedence constraints are fulfilled, i.e., $S_i + p_i \leq S_j$ if $i \rightarrow j$, and
- the **makespan** $C_{\max} = \max_{j=1}^n \{C_j\}$ is minimized, where $C_j := S_j + p_j$ is assumed to be the completion time of activity j .

The fact that an activity which starts at time S_j finishes at time $C_j = S_j + p_j$ implies that activities are not preempted. We may relax this condition by allowing **preemption** (activity splitting). In this case the processing of any activity may be interrupted and resumed later.

It is often useful to add a unique **dummy starting activity** 0 and a unique **dummy termination activity** $n + 1$, which indicate the start and the end

of the project, respectively. The dummy activities need no resources and have processing time 0. In order to impose $0 \rightarrow j \rightarrow n+1$ for all activities $j = 1, \dots, n$ we set $0 \rightarrow j$ for all activities j without any predecessor and $j \rightarrow n+1$ for all activities j without any successor. Then S_0 is the starting time of the project and $S_{n+1} - S_0$ may be interpreted as the makespan of the project. Usually we set $S_0 := 0$.

If preemption is not allowed, the vector $S = (S_j)$ defines a **schedule** of the project. S is called **feasible** if all resource and precedence constraints are fulfilled.

Example 1 Consider a project with $n = 4$ activities, $r = 2$ resources with capacities $R_1 = 5, R_2 = 7$, a precedence relation $2 \rightarrow 3$ and the following data:

j	1	2	3	4
p_j	4	3	5	8
r_{j1}	2	1	2	2
r_{j2}	3	5	2	4

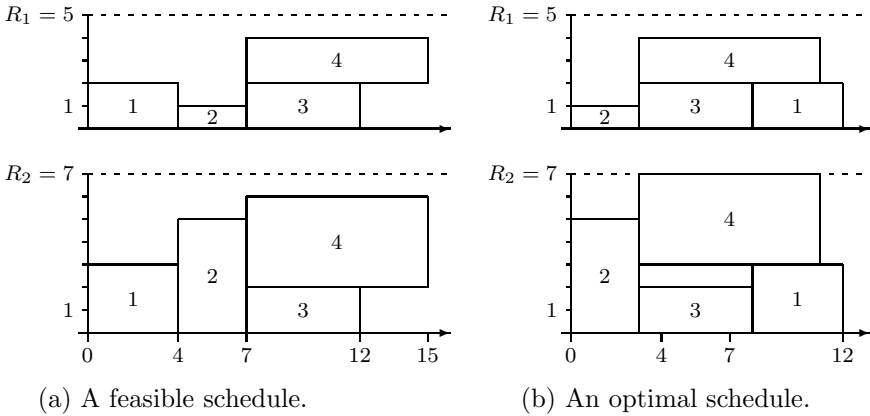


FIGURE 1.1: Two feasible schedules for Example 1.

In Figure 1.1(a) a so-called **Gantt chart** of a feasible schedule with $C_{\max} = 15$ is drawn. This schedule does not minimize the makespan, since by moving activity 1 to the right, a shorter schedule is obtained. An optimal schedule with makespan $C_{\max} = 12$ is shown in (b).

A precedence relation $i \rightarrow j$ with the meaning $S_i + p_i \leq S_j$ may be generalized by a start-start relation of the form

$$S_i + d_{ij} \leq S_j \quad (1.1)$$

with an arbitrary integer number $d_{ij} \in \mathbb{Z}$. The interpretation of relation (1.1) depends on the sign of d_{ij} :

- If $d_{ij} \geq 0$, then activity j cannot start before d_{ij} time units after the start of activity i . This means that activity j does not start before activity i and d_{ij} is a minimal distance (time-lag) between both starting times (cf. Figure 1.2(a)).
- If $d_{ij} < 0$, then the earliest start of activity j is $-d_{ij}$ time units before the start of activity i , i.e., activity i cannot start more than $-d_{ij}$ time units later than activity j . If $S_j \leq S_i$, this means that $|d_{ij}|$ is a maximal distance between both starting times (cf. Figure 1.2(b)).



FIGURE 1.2: Positive and negative time-lags.

If $d_{ij} > 0$ holds, the value is also called a **positive time-lag**; if $d_{ij} < 0$, it is called a **negative time-lag**.

Relations (1.1) are very general timing relations between activities. For example, (1.1) with $d_{ij} = p_i$ is equivalent to the precedence relation $i \rightarrow j$. More generally, besides start-start relations, finish-start, finish-finish, or start-finish relations may be considered. But if no preemption is allowed, any type of these relations can be transformed to any other type. For example, finish-start relations $C_i + l_{ij} \leq S_j$ with finish-start time-lags l_{ij} can be transformed into start-start relations $S_i + p_i + l_{ij} \leq S_j$ by setting $d_{ij} := p_i + l_{ij}$.

Generalized precedence relations may, for example, be used in order to model certain timing restrictions for a chemical process. If $S_i + p_i + l_{ij} \leq S_j$ and $S_j - u_{ij} - p_i \leq S_i$ with $0 \leq l_{ij} \leq u_{ij}$ are required, then the time between the completion time of activity i and the starting time of activity j must be at least l_{ij} but no more than u_{ij} . This includes the special case $0 \leq l_{ij} = u_{ij}$ where activity j must start exactly l_{ij} time units after the completion of activity i . If $l_{ij} = u_{ij} = 0$, then activity j must start immediately after activity i finishes (**no-wait constraint**).

Also release times r_j and deadlines d_j of activities j can be modeled by relations of the form (1.1). While a **release time** r_j is an earliest starting time for activity j , a **deadline** d_j is a latest completion time for j . To model release times we add the restrictions $S_0 + r_j \leq S_j$. To model deadlines we add the restrictions $S_j - (d_j - p_j) \leq S_0$. In both cases we assume that $S_0 = 0$.

If $r_j \leq d_j$ for a release time r_j and a deadline d_j , then the interval $[r_j, d_j]$ is called a **time window** for activity j . Activity j must be processed completely within its time window.

In processor scheduling sometimes also so-called **communication delays** $c_{ij} > 0$ are considered. If two activities i, j with $i \rightarrow j$ are processed on the same processor, no communication overhead arises and the constraint is interpreted as a usual precedence relation $S_i + p_i \leq S_j$. If, on the other hand, i, j are processed on different processors, a communication delay occurs which means that $S_i + p_i + c_{ij} \leq S_j$ has to be satisfied.

In the **multi-mode case** a set $\mathcal{M}_j$ of so-called modes (processing alternatives) is associated with each activity j . The processing time of activity j in mode m is given by p_{jm} and the per period usage of a renewable resource k is given by r_{jkm} . One has to assign a mode to each activity and to schedule the activities in the assigned modes. Multiple modes may, for example, be used in order to model a situation in which an activity can be processed quickly by many workers or more slowly with less people.

In the multi-mode situation, besides renewable resources like processors, machines, or people we may also have so-called **non-renewable resources** like money or energy. While renewable resources are available with a constant amount in each time period again, the availability of non-renewable resources is limited for the whole time horizon of the project. This means that non-renewable resources are consumed, i.e., when an activity j is processed, the available amount R_k of a non-renewable resource k is decreased by r_{jk} .

Non-renewable resources are only important in connection with multi-mode problems, because in the single-mode case the resource requirements of non-renewable resources are schedule independent (the available non-renewable resources may be sufficient or not). On the other hand, in the multi-mode case the resource requirements of non-renewable resources depend on the choice of modes (i.e., feasible and infeasible mode assignments may exist).

Besides the objective of minimizing the makespan $C_{\max} := \max_{j=1}^n \{C_j\}$ one may consider other objective functions $f(C_1, \dots, C_n)$ depending on the completion times of the activities. Examples are the **total flow time** $\sum_{j=1}^n C_j$ or more generally the **weighted (total) flow time** $\sum_{j=1}^n w_j C_j$ with non-negative weights $w_j \geq 0$.

Other objective functions depend on **due dates** d_j that are associated with the activities. With the **lateness** $L_j := C_j - d_j$, the **tardiness** $T_j := \max\{0, C_j - d_j\}$, and the **unit penalty** $U_j := \begin{cases} 0, & \text{if } C_j \leq d_j \\ 1, & \text{otherwise} \end{cases}$ the following

objective functions are common:

the maximum lateness	$L_{\max} := \max_{j=1}^n L_j$
the total tardiness	$\sum_{j=1}^n T_j$
the total weighted tardiness	$\sum_{j=1}^n w_j T_j$
the number of late activities	$\sum_{j=1}^n U_j$
the weighted number of late activities	$\sum_{j=1}^n w_j U_j.$

All these objective functions f are monotone non-decreasing in the completion times C_j , i.e., they satisfy $f(C_1, \dots, C_n) \leq f(C'_1, \dots, C'_n)$ for completion time vectors C, C' with $C_j \leq C'_j$ for all $j = 1, \dots, n$. They are also called **regular**.

1.3 Processor (Machine) Scheduling

In this section we introduce some important classes of processor (machine) scheduling problems (see also Błażewicz et al. [7], Brucker [8]).

Single processor scheduling

In the simplest scheduling model we are given n jobs $J_1, \dots, J_n$ with processing times p_j ($j = 1, \dots, n$) which have to be processed on a single processor. Additionally, precedence constraints may be given. Such a problem can be modeled as an RCPSP with one renewable resource with unit capacity where all jobs require one unit of the resource.

Example 2 Consider an instance with $n = 5$ jobs, processing times $p_1 = 3$, $p_2 = 2$, $p_3 = 4$, $p_4 = 2$, $p_5 = 3$ and precedence constraints $1 \rightarrow 3$, $2 \rightarrow 4$, $4 \rightarrow 5$. A feasible schedule for this instance with makespan $C_{\max} = 14$ is shown in Figure 1.3.

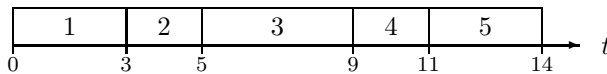


FIGURE 1.3: Single processor schedule.

Parallel processor scheduling

Instead of a single processor we may have m processors $P_1, \dots, P_m$ on which the jobs have to be processed. If the processors are **identical**, the processing time p_j of job J_j does not depend on the processor on which J_j is processed. This problem corresponds to an RCPSP with one cumulative resource with capacity m where all jobs require one unit of the resource.

Example 3 Consider an instance with $n = 8$ jobs, $m = 3$ processors, processing times $p_1 = 1, p_2 = 3, p_3 = 4, p_4 = 2, p_5 = 2, p_6 = 3, p_7 = 1, p_8 = 5$, and precedence constraints as shown in the left part of Figure 1.4. A feasible schedule with makespan $C_{\max} = 9$ is shown in the right part of the figure.

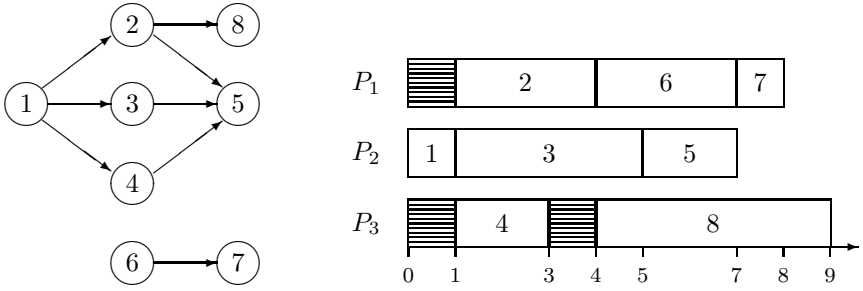


FIGURE 1.4: Schedule for identical parallel processors.

By contrast with the case with identical processors, for so-called **unrelated** processors the processing time p_{jk} of job J_j depends on the processor P_k ($k = 1, \dots, m$) on which J_j is processed. The processors are called **uniform** if $p_{jk} = p_j/s_k$ where s_k may be interpreted as the speed of processor P_k . Problems with unrelated processors can be modeled as a multi-mode RCPSP with m renewable resources where each resource has unit capacity. Each job J_j has m modes corresponding to the processors on which J_j may be processed. If job J_j is processed in mode k , then p_{jk} is its processing time and J_j uses only one unit of resource k (processor P_k).

A further generalization are scheduling problems with **multi-purpose processors** (flexible machines). In this situation we associate with each job J_j a subset of processors $\mu_j \subseteq \{P_1, \dots, P_m\}$ indicating that J_j can be executed by any processor of this set. If job J_j is processed on processor P_k , then its processing time is equal to p_{jk} (or simply to p_j if the processing time does not depend on the assigned processor). This problem can be formulated as above as a multi-mode RCPSP with m renewable resources where each job J_j has only $|\mu_j|$ modes corresponding to the processors on which J_j may be processed.

Multi-processor task scheduling

In a **multi-processor task scheduling problem** we have n jobs $J_1, \dots, J_n$ and m processors $P_1, \dots, P_m$. Associated with each job J_j is a processing time p_j and a subset of processors $\mu_j \subseteq \{P_1, \dots, P_m\}$. During its processing job J_j occupies each of the processors in μ_j . Finally, precedence constraints may be given between certain jobs.

This problem can be formulated as an RCPSP with m renewable resources where each resource has unit capacity. Furthermore, job J_j requires one unit of all processors $P_k \in \mu_j$ simultaneously.

Example 4 Consider the following instance with $n = 5$ jobs and $m = 3$ processors:

j	1	2	3	4	5
μ_j	$\{P_1, P_2\}$	$\{P_2, P_3\}$	$\{P_1, P_2\}$	$\{P_3\}$	$\{P_1, P_2, P_3\}$
p_j	1	2	2	3	1

In Figure 1.5 a feasible schedule with makespan $C_{\max} = 7$ for this instance is shown. It does not minimize the makespan since by processing job J_1 together with job J_4 we can get a schedule with $C_{\max} = 6$.

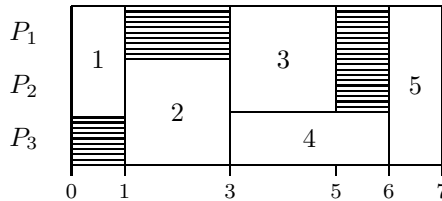


FIGURE 1.5: Feasible schedule for a multi-processor task problem.

In another variant of multi-processor task problems all processors are assumed to be identical and each job can be processed by any processor. With each job J_j an integer number $size_j$ is associated with the meaning that during its processing J_j needs $size_j$ processors simultaneously.

Shop scheduling

In so-called shop scheduling problems the jobs consist of several operations which have to be processed on different processors. In **general-shop scheduling problems** we have jobs $J_1, \dots, J_n$ and m processors $P_1, \dots, P_m$. Job J_j consists of n_j operations $O_{1j}, \dots, O_{n_j,j}$. Two operations of the same

job cannot be processed at the same time and a processor can process at most one operation at any time. Operation O_{ij} must be processed for p_{ij} time units on a dedicated processor from the set $\{P_1, \dots, P_m\}$. Furthermore, precedence constraints may be given between arbitrary operations.

Such a general-shop scheduling problem can be modeled as an RCPSP with $m + n$ renewable resources where each resource has unit capacity. While the resources $k = 1, \dots, m$ correspond to the processors, the resources $m + j$ ($j = 1, \dots, n$) are needed to model the fact that different operations of job J_j cannot be processed at the same time. Furthermore, we have $\sum_{j=1}^n n_j$ activities O_{ij} , where operation O_{ij} needs one unit of the “processor resource” corresponding to the dedicated processor of O_{ij} and one unit of the “job resource” $m + j$.

Important special cases of the general-shop scheduling problem are job-shop, flow-shop, and open-shop problems, which will be discussed next.

A **job-shop problem** is a general-shop scheduling problem with chain precedences of the form

$$O_{1j} \rightarrow O_{2j} \rightarrow \dots \rightarrow O_{n_j,j}$$

for $j = 1, \dots, n$ (i.e., there are no precedences between operations of different jobs and the precedences between operations of the same job build a chain). Note that for a job-shop problem no “job resource” is needed, since all operations of the same job are linked by a precedence relation (and thus cannot be processed simultaneously).

A **flow-shop problem** is a special job-shop problem with $n_j = m$ operations for $j = 1, \dots, n$ where operation O_{ij} must be processed on P_i . In a so-called **permutation flow-shop problem** the jobs have to be processed in the same order on all processors.

Example 5 In Figure 1.6 a feasible schedule for a permutation flow-shop problem with $n = 4$ jobs and $m = 3$ processors is shown.

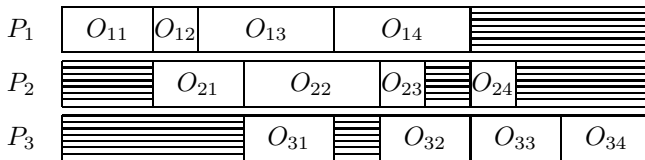


FIGURE 1.6: Permutation schedule for a flow-shop problem.

An **open-shop problem** is like a flow-shop problem but without any precedence relations between the operations. Thus, it also has to be decided in

which order the operations of a job are processed.

Example 6 In Figure 1.7 a feasible schedule for an open-shop problem with $n = 2$ jobs and $m = 3$ processors is shown. In this schedule the operations of job J_1 are processed in the order $O_{11} \rightarrow O_{31} \rightarrow O_{21}$, the operations of job J_2 are processed in the order $O_{32} \rightarrow O_{22} \rightarrow O_{12}$.

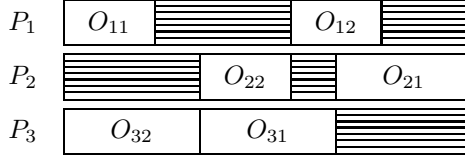


FIGURE 1.7: Feasible schedule for an open-shop problem.

A classification scheme

Classes of scheduling problems are often specified in terms of a three-field classification $\alpha|\beta|\gamma$ introduced by Graham et al. [12]. The first entry α specifies the **processor environment**, β specifies **job characteristics**, and γ denotes an **optimality criterion** (objective function).

The first entry α specifies the processor environment, where 1 stands for a single processor, P denotes parallel identical, Q uniform, and R unrelated processors. For open-shop, flow-shop, and job-shop problems the symbols O , F , and J are used. If the number m of processors is fixed (i.e., not part of the input), this number m is added to the α -field (e.g., $P3$ specifies a problem with three identical processors, Pm denotes a parallel processor problem with a fixed number m of processors, and $F2$ denotes a two-processor flow-shop problem).

The β -field specifies job characteristics concerning processing times, release dates, precedences, etc. The entry $p_j = p$ ($p_{ij} = p$) denotes that all jobs (operations) have constant processing times and $p_j = 1$ denotes that they are all equal to one. Furthermore, r_j indicates release dates for the jobs and $pmtn$ denotes that preemption of the jobs is allowed. If precedence relations are given, the entry $prec$ is used (or *chains*, *tree*, *sp-graph* if the precedences have a special structure). Additionally, in a multi-processor task environment with parallel processors, the entry $size_j$ says that a job needs several processors simultaneously. Communication delays are indicated by an entry c_{ij} (or $c_{ij} = c$, $c_{ij} = 1$ for constant and unit communication delays, respectively).

Finally, γ specifies the objective function. It is set to $C_{\max}$, $L_{\max}$, $\sum C_j$,

$\sum w_j C_j$, $\sum U_j$, $\sum w_j U_j$, $\sum T_j$, $\sum w_j T_j$ to denote one of the objective functions introduced in Section 1.2.

In order to illustrate the $\alpha|\beta|\gamma$ -notation three examples are presented:

- $1 \mid r_j; pmtn \mid L_{\max}$ denotes a problem with a single processor, where the jobs have release dates, preemption is allowed, and the maximum lateness has to be minimized.
- $P2 \mid p_j = p; r_j; tree \mid C_{\max}$ denotes a parallel processor problem with two processors, where the jobs have release dates and the same processing time p , precedence constraints in form of a tree are given and the makespan has to be minimized.
- $Jm \mid p_{ij} = 1 \mid \sum w_j U_j$ denotes a job-shop problem with a fixed number of processors, where all operations have unit processing times and the weighted number of late jobs has to be minimized.

1.4 Easy and Hard Problems

When we consider scheduling problems (or more generally combinatorial optimization problems) an important issue is the complexity of a problem. For a new problem we often first try to develop an algorithm that solves the problem in an efficient way. If we cannot find such an algorithm, it may be helpful to prove that the problem is NP-hard, which implies that most likely no efficient algorithm exists. In this section we review the most important aspects of complexity theory (cf. also Garey and Johnson [11]) which allow us to decide whether a problem is “easy” or “hard”.

A computational problem can be viewed as a function h which maps each input x in some given domain to an output $h(x)$ in some given range. Of interest are algorithms which solve computational problems, i.e., compute $h(x)$ for each input x . The length $|x|$ of an input x is defined as the length of some encoding of x . The efficiency of an algorithm may be measured by its running time, i.e., the number of steps it needs for a certain input. Since it is reasonable that this number is larger for a larger input, an efficiency function should depend on the input size $|x|$. The size of an input for a computer program is usually defined by the length of a **binary encoding** of the input. For example, in a binary encoding an integer number a is represented as a binary number using $\log_2 a$ bits, an array with m numbers needs $m \log_2 a$ bits when a is the largest number in the array. On the other hand, in a so-called **unary encoding** a number a is represented by a bits.

Since in most cases it is very difficult to determine the average running time of an algorithm for an input with size n , often the worst-case running time of an algorithm is studied. For this purpose, the function $T(n)$ may be

defined as an upper bound on the running time of the algorithm on any input x with size $|x| = n$. Since often it is also difficult to give a precise description of $T(n)$, we only consider the growth rate of $T(n)$ which is determined by its asymptotic order. We say that a function $T(n)$ is $O(g(n))$ if constants $c > 0, n_0 \in \mathbb{N}$ exist such that $T(n) \leq cg(n)$ for all $n \geq n_0$. For example, the function $T_1(n) = 37n^3 + 4n^2 + n$ is $O(n^3)$, the function $T_2(n) = 2^n + n^{100} + 4$ is $O(2^n)$.

If the running time of an algorithm is bounded by a polynomial function in n , i.e., if it is $O(n^k)$ for some constant $k \in \mathbb{N}$, the algorithm is called a **polynomial-time** algorithm. If the running time is bounded by a polynomial function in the input size of a unary encoding, an algorithm is called **pseudo-polynomial**. For example, an algorithm with running time $O(n^2 a)$ is pseudo-polynomial if the input size of a binary encoding is $O(n \log a)$.

As an example for a polynomial-time algorithm we consider the scheduling problem $1||\sum w_j C_j$.

Example 7 *The input x of problem $1||\sum w_j C_j$ consists of the number n of jobs and two n -dimensional vectors (p_j) and (w_j) . Thus, the input length $|x|$ in a binary encoding is given by $|x| = n \log_2 a$ where $a := \max_{j=1}^n \{p_j, w_j\}$ denotes the largest numeric value in the input. The output $h(x)$ for this problem is a schedule S minimizing $\sum_{j=1}^n w_j C_j$. It can be represented by a vector of the completion times C_j for all jobs J_j . The following algorithm of Smith [27], also called Smith's weighted shortest processing time first (WSPT) rule, calculates optimal C_j -values:*

Algorithm 1 $||\sum w_j C_j$

1. Enumerate the jobs such that $w_1/p_1 \geq w_2/p_2 \geq \dots \geq w_n/p_n$;
2. $C_0 := 0$;
3. FOR $j := 1$ TO n DO
 $C_j := C_{j-1} + p_j$;

The number of computational steps in this algorithm can be bounded as follows. In Step 1 the jobs have to be sorted, which takes $O(n \log n)$ steps. Furthermore, Step 3 can be done in $O(n)$ time. Thus, $T(n) \in O(n \log n)$. If n is replaced by the input length $|x| = n \log_2 a \geq n$, the bound is still valid.

Finally, we show that each schedule constructed by the algorithm is optimal. Clearly, in an optimal schedule starting at time zero the jobs are scheduled one after the other without idle times on the machine. It has to be shown that scheduling the jobs in an order of non-increasing ratios $\frac{w_j}{p_j}$ is optimal. This can be done by proving that if in a schedule S a job J_i is scheduled immediately before a job J_j with $\frac{w_i}{p_i} \leq \frac{w_j}{p_j}$ (or, equivalently, $p_j w_i \leq p_i w_j$), swapping J_i and J_j does not increase the objective value. Let S_i be the starting time of job J_i in S . Then, the contribution of the jobs J_i, J_j in S to the objective value $\sum w_j C_j$ is $c(S) := (S_i + p_i)w_i + (S_i + p_i + p_j)w_j$. Let S' be the schedule

derived from S by swapping J_i and J_j . The contribution of the jobs J_i, J_j in S' to the objective value is $c(S') := (S_i + p_j)w_j + (S_i + p_j + p_i)w_i$, which is not greater than $c(S)$.

To see that a schedule constructed by the algorithm is optimal assume that the jobs are enumerated as in Step 1 and consider an arbitrary optimal schedule S . If in S job J_1 is not scheduled as the first job, J_1 can be swapped with its predecessors until it is scheduled in the first position. Due to $\frac{w_1}{p_1} \geq \frac{w_i}{p_j}$ for all jobs J_j , the objective value does not increase. Next, job J_2 will be moved to the second position, etc. Finally, the schedule constructed by the algorithm is reached, which has the same objective value as the optimal schedule S .

Since exponential functions grow much faster than polynomial functions, exponential-time algorithms cannot be used for larger problems. If we compare two algorithms with running times $O(n^3)$ and $O(2^n)$ under the assumption that a computer can perform 10^9 steps per second, we get the following result: while for $n = 1000$ the first algorithm is still finished after one second, the second algorithm needs already 18 minutes for $n = 40$, 36 years for $n = 60$ and even 374 centuries for $n = 70$.

For this reason a problem is classified as “easy” (tractable) if it can be solved by a polynomial-time algorithm. In order to classify a problem as “hard” (intractable), the notion of NP-hardness was developed which will be discussed next.

A problem is called a **decision problem** if its answer is only “yes” or “no”. An instance for which the answer is “yes” is also called a “yes”-instance, an instance for which the answer is “no” a “no”-instance. A famous example for a decision problem is the so-called **partition problem**: Given n positive integers $a_1, \dots, a_n$ and the value $b := \frac{1}{2} \sum_{i=1}^n a_i$, is there a subset $A \subset \{1, \dots, n\}$ such that $\sum_{i \in A} a_i = \sum_{i \notin A} a_i = b$?

On the other hand, most scheduling problems belong to the class of **optimization problems**, where the answer is a feasible solution which minimizes (or maximizes) a given objective function c . For each scheduling problem a corresponding decision problem may be defined by asking whether there exists a feasible schedule S with $c(S) \leq y$ for a given threshold value y .

The class of all polynomially solvable decision problems is denoted by $\mathcal{P}$. Another important complexity class is the set $\mathcal{NP}$ which contains all decision problems that are nondeterministically polynomially solvable. An equivalent definition of this class is based on so-called certificates which can be verified in polynomial time.

A **certificate** for a “yes”-instance of a decision problem is a piece of information which proves that a “yes”-answer for this instance exists. For example, for the partition problem a set A with $\sum_{i \in A} a_i = b$ is a certificate for a “yes”-instance. For a decision problem corresponding to a scheduling problem a feasible schedule S with $c(S) \leq y$ provides a certificate for a “yes”-instance.

If such a certificate is short and can be checked in polynomial time, the corresponding decision problem belongs to the class $\mathcal{NP}$. More precisely, the set $\mathcal{NP}$ contains all decision problems where each “yes”-instance I has a certificate which

- has a length polynomially bounded in the input size of I , and
- can be verified by a polynomial-time algorithm.

Note that there is an important asymmetry between “yes”- and “no”-instances. While a “yes”-answer can easily be certified by a single feasible schedule S with $c(S) \leq y$, a “no”-answer cannot be certified by a short piece of information.

It is easy to see that every polynomially solvable decision problem belongs to $\mathcal{NP}$ (the output of the polynomial-time algorithm may be used as a certificate which can also be verified by the algorithm in polynomial time). Thus, $\mathcal{P} \subseteq \mathcal{NP}$. One of the most challenging open problems in theoretical computer science today is the question whether $\mathcal{P} = \mathcal{NP}$ holds. Although it is generally conjectured that $\mathcal{P} \neq \mathcal{NP}$ holds, up to now nobody was able to prove this conjecture.

The central issue for the definition of NP-hardness is the notion of a polynomial **reduction**. A decision problem P is said to be **polynomially reducible** to another decision problem Q (denoted by $P \propto Q$) if a polynomial-time computable function g exists which transforms every instance I for P into an instance $I' = g(I)$ for Q such that I is a “yes”-instance for P if and only if I' is a “yes”-instance for Q .

A polynomial reduction $P \propto Q$ between two decision problems P, Q implies that Q is at least as difficult as P . This means that if Q is polynomially solvable, then also P is polynomially solvable. Equivalently, if P is not polynomially solvable, then also Q cannot be solved in polynomial time. Furthermore, reducibility is transitive, i.e., if $P \propto Q$ and $Q \propto R$ for three decision problems P, Q, R , then also $P \propto R$.

A decision problem P is called **NP-complete** if

- (i) P belongs to the class $\mathcal{NP}$, and
- (ii) every other decision problem $Q \in \mathcal{NP}$ is polynomially reducible to P .

A problem P is called **NP-hard** if only (ii) holds. Especially, an optimization problem is NP-hard if the corresponding decision problem is NP-complete.

If any single NP-complete problem could be solved in polynomial time, then all problems in $\mathcal{NP}$ would also be polynomially solvable, i.e., we would have $\mathcal{P} = \mathcal{NP}$. Since nobody has found a polynomial-time algorithm for any NP-complete problem yet, most likely no polynomial-time algorithm exists for these problems.

An extension of NP-completeness is the concept of strongly NP-complete problems. If a problem P is **strongly NP-complete**, then it is unlikely that

it can be solved by a pseudo-polynomial algorithm, since this would imply $\mathcal{P} = \mathcal{NP}$.

In 1971 Steve Cook was the first to prove that NP-complete problems exist. He showed that the so-called **satisfiability problem** (in which it has to be decided whether a given boolean formula can be satisfied or not) is NP-complete. This result together with the transitivity of reducibility implies that for an NP-completeness proof of a decision problem $P \in \mathcal{NP}$ it is sufficient to show that

- (ii') **one** known NP-complete problem $Q \in \mathcal{NP}$ is polynomially reducible to P

(since then we have $R \propto P$ for **all** problems $R \in \mathcal{NP}$ due to $R \propto Q \propto P$).

In this way the partition problem and other fundamental combinatorial problems have been shown to be NP-complete by reductions from the satisfiability problem. Usually, it is easy to show that P belongs to the class $\mathcal{NP}$. Thus, the main task in proving NP-completeness of a problem P is to find a known NP-complete problem Q which can be reduced to P .

The following example shows a reduction from the partition problem to the three-machine flow-shop scheduling problem $F3||C_{\max}$.

Example 8 *The partition problem is reducible to the decision version of problem $F3||C_{\max}$. To prove this reduction, consider a three-machine flow-shop problem with $n + 1$ jobs $J_1, \dots, J_{n+1}$, where the processing times of the operations are given by*

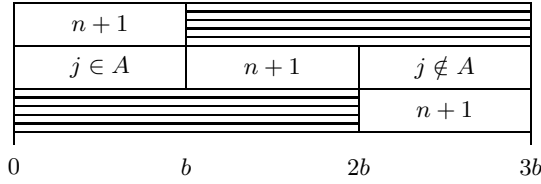
$$\begin{aligned} p_{1j} &:= 0, \quad p_{2j} := a_j, \quad p_{3j} := 0 \quad \text{for } j = 1, \dots, n; \\ p_{1,n+1} &= p_{2,n+1} = p_{3,n+1} := b = \frac{1}{2} \sum_{j=1}^n a_j. \end{aligned}$$

and the threshold for the corresponding decision problem is set to $y := 3b$. Obviously, the input of the flow-shop problem can be computed in polynomial time from the input of the partition problem.

If the partition problem has a solution, then an index set $A \subseteq \{1, \dots, n\}$ exists such that $\sum_{j \in A} a_j = b$. In this case the schedule shown in Figure 1.8 solves the decision version of problem $F3||C_{\max}$.

If, on the other hand, the flow-shop problem has a solution with $C_{\max} \leq 3b$, then job $n + 1$ must be scheduled as shown in Figure 1.8. Furthermore, the set $A = \{j \mid \text{job } J_j \text{ finishes not later than } b\}$ solves the partition problem.

Many reductions exist between scheduling problems. For example, problems with unit processing times $p_j = 1$ are a special case of constant processing times $p_j = p$ which can be reduced to the general case of arbitrary processing times p_j . Problems without release dates may be reduced to problems with arbitrary release dates by setting $r_j = 0$ for all j . Furthermore, problem $1||\sum C_j$ reduces to $1||\sum w_j C_j$ because any instance of $1||\sum C_j$ defines also

FIGURE 1.8: Solution of $F3||C_{\max}$.

an instance of $1||\sum w_j C_j$ by setting $w_j := 1$ for all $j = 1, \dots, n$. These reductions are called elementary.

Other elementary reductions for objective functions can be found in Figure 1.9. In it we use the results that $\sum f_j$ reduces to $\sum w_j f_j$ by setting $w_j = 1$ for all j . Furthermore, $C_{\max}$, $\sum C_j$, and $\sum w_j C_j$ reduce to $L_{\max}$, $\sum T_j$, and $\sum w_j T_j$, respectively, by setting $d_j = 0$ for all j .

Finally, for any threshold value y we have

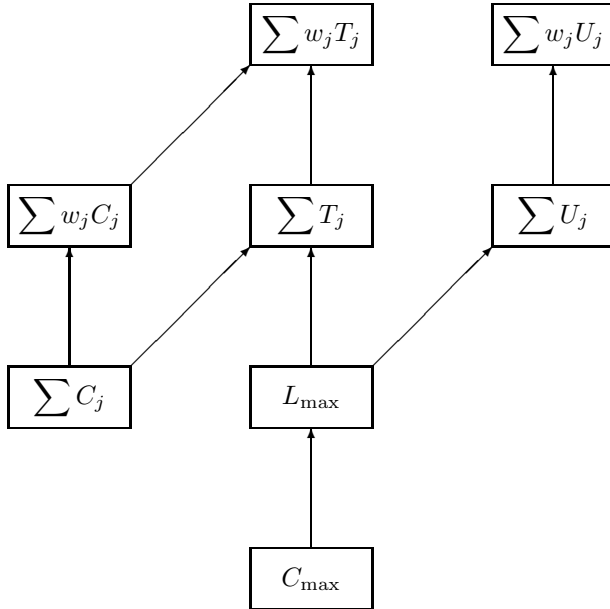


FIGURE 1.9: Elementary reductions for objective functions.

$$\begin{aligned}
L_{\max} \leq y &\Leftrightarrow C_j - d_j \leq y \text{ for all } j \\
&\Leftrightarrow C_j - (d_j + y) \leq 0 \text{ for all } j \\
&\Leftrightarrow \max\{0, C_j - (d_j + y)\} \leq 0 \text{ for all } j \\
&\Leftrightarrow \sum T_j = \sum \max\{0, C_j - (d_j + y)\} \leq 0 \\
&\Leftrightarrow \sum U_j \leq 0.
\end{aligned}$$

Thus, $L_{\max}$ reduces to $\sum T_j$ and $\sum U_j$.

For a single processor in the case of a regular objective function it can be shown that if no release dates are given or if all processing times are equal to one, allowing preemption does not lead to better solutions (see e.g., Brucker [8]). Thus, for a single processor we get reductions between characteristics in the β -field as shown in Figure 1.10. Note that the symbol $\circ$ denotes that the corresponding entry is empty, i.e., no precedence constraints in Figure 1.10(a); no release dates, no preemption allowed, and arbitrary processing times in (b).

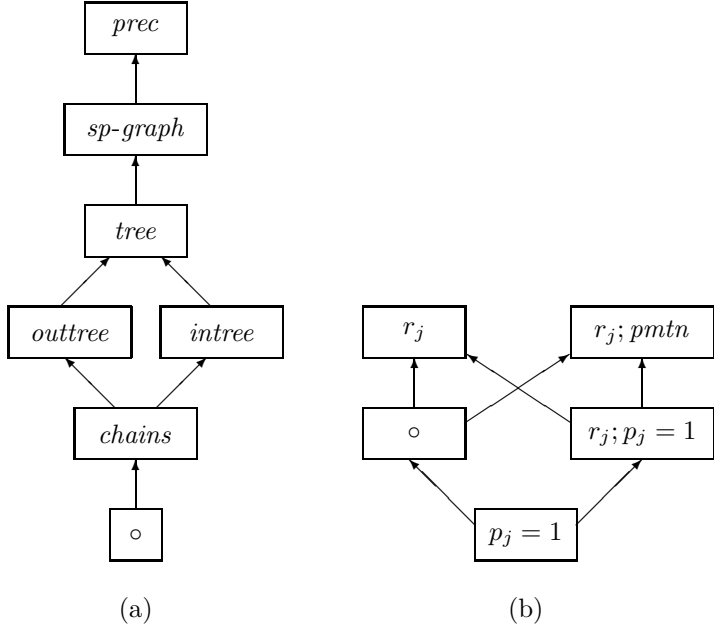


FIGURE 1.10: Reductions for single processor problems.

1.5 Complexity Classification of Scheduling Problems

In this section all known complexity results for the class of single processor problems are presented. We distinguish between “easy” and “hard” problems which correspond to problems which are polynomially solvable and NP-hard, respectively.

Of interest are the hardest problems in the class which have been shown to be polynomially solvable, the simplest problems which are already known to be NP-hard, as well as the simplest and hardest problems for which the complexity status is unknown.

Based on the elementary reductions (see Figures 1.9 and 1.10), known polynomial-time algorithms and known NP-hardness results in Table 1.1 all complexity results for problems with a single processor are summarized. We distinguish the following subclasses:

- Maximal polynomially solvable problems: A problem P is maximal polynomially solvable if no elementary reduction from P to another problem Q exists, where Q has been shown to be polynomially solvable. For example, problem $1||\sum w_j C_j$ is not maximal polynomially solvable since its generalization $1|sp-graph|\sum w_j C_j$ is still polynomially solvable.
- Minimal NP-hard problems: A problem P is minimal NP-hard if no elementary reduction from another problem Q to P exists, where Q has been shown to be NP-hard. For example, problem $1|r_j|\sum w_j T_j$ is not minimal NP-hard since its special case $1|r_j|\sum C_j$ is already NP-hard. Problems that are strongly NP-hard are marked with *.
- Minimal and maximal open problems.

Results and corresponding literature references for other classes of processor scheduling problems (parallel processor problems, shop problems, multi-processor task problems, and several generalized problems) can be found at our website:

<http://www.mathematik.uni-osnabrueck.de/research/OR/class/>.

maximal polynomially solvable:	
$1 prec; r_j C_{\max}$	Lawler [15]
$1 prec; p_j = p; r_j L_{\max}$	Simons [25]
$1 prec; r_j; pmtn L_{\max}$	Błażewicz [6], Baker et al. [2]
$1 prec; p_j = p; r_j \sum C_j$	Simons [26]
$1 prec; pmtn; p_j = p; r_j \sum C_j$	Baptiste et al. [5]
$1 r_j; pmtn \sum C_j$	Baker [1]
$1 p_j = p; r_j \sum w_j C_j$	Baptiste [4]
$1 sp-graph \sum w_j C_j$	Lawler [17]
$1 r_j; pmtn \sum U_j$	Lawler [18]
$1 p_j = p; r_j \sum w_j U_j$	Baptiste [3]
$1 pmtn; p_j = p; r_j \sum w_j U_j$	Baptiste [3]
$1 p_j = p; r_j \sum T_j$	Baptiste [4]
$1 pmtn; p_j = p; r_j \sum T_j$	Tian et al. [28]
$1 p_j = 1; r_j \sum w_j T_j$	Assignment problem

minimal NP-hard:	
* $1 r_j L_{\max}$	Lenstra et al. [23]
* $1 chains; r_j; pmtn \sum C_j$	Lenstra [20]
* $1 prec \sum C_j$	Lawler [17], Lenstra and Rinnooy Kan [21]
* $1 r_j \sum C_j$	Lenstra et al. [23]
* $1 chains; p_j = 1; r_j \sum w_j C_j$	Lenstra and Rinnooy Kan [22]
* $1 prec; p_j = 1 \sum w_j C_j$	Lawler [17], Lenstra and Rinnooy Kan [21]
* $1 r_j; pmtn \sum w_j C_j$	Labetoulle et al. [14]
* $1 chains; p_j = 1 \sum U_j$	Lenstra and Rinnooy Kan [22]
$1 \sum w_j U_j$	Lawler and Moore [19], Karp [13]
$1 \sum T_j$	Lawler [16], Du and Leung [10]
* $1 chains; p_j = 1 \sum T_j$	Leung and Young [24]
* $1 \sum w_j T_j$	Lawler [16], Lenstra et al. [23]

minimal open:	maximal open:
$1 pmtn; p_j = p; r_j \sum w_j C_j$	$1 pmtn; p_j = p; r_j \sum w_j T_j$
$1 p_j = p; r_j \sum w_j T_j$	$1 p_j = p; r_j \sum w_j T_j$

Table 1.1: Complexity results for problems with a single processor.

References

- [1] K. Baker. *Introduction to Sequencing and Scheduling*. John Wiley & Sons, New York, 1974.
- [2] K. Baker, E. Lawler, J. Lenstra, and A. Rinnooy Kan. Preemptive scheduling of a single machine to minimize maximum cost subject to release dates and precedence constraints. *Operations Research*, 31:381–386, 1983.
- [3] P. Baptiste. Polynomial time algorithms for minimizing the weighted number of late jobs on a single machine with equal processing times. *Journal of Scheduling*, 2:245–252, 1999.
- [4] P. Baptiste. Scheduling equal-length jobs on identical parallel machines. *Discrete Applied Mathematics*, 103(1):21–32, 2000.
- [5] P. Baptiste, P. Brucker, S. Knust, and V. Timkovsky. Ten notes on equal-execution-time scheduling. *JOR*, 2:111–127, 2004.
- [6] J. Błażewicz. Scheduling dependent tasks with different arrival times to meet deadlines. In *Model. Perform. Eval. Comput. Syst., Proc. int. Workshop, Stresa 1976*, pages 57–65. North Holland, Amsterdam, 1976.
- [7] J. Błażewicz, K. Ecker, E. Pesch, G. Schmidt, and J. Węglarz. *Scheduling Computer and Manufacturing Processes*. Springer, Berlin, 2nd edition, 2001.
- [8] P. Brucker. *Scheduling Algorithms*. Springer, Berlin, 5th edition, 2007.
- [9] P. Brucker and S. Knust. *Complex Scheduling*. Springer, Berlin, 2006.
- [10] J. Du and J.-T. Leung. Minimizing total tardiness on one machine is NP-hard. *Mathematics of Operations Research*, 15(3):483–495, 1990.
- [11] M. R. Garey and D. S. Johnson. *Computers and Intractability. A Guide to the Theory of NP-Completeness*. W. H. Freeman & Co, San Francisco, 1979.
- [12] R. Graham, E. Lawler, J. Lenstra, and A. Rinnooy Kan. Optimization and approximation in deterministic sequencing and scheduling: A survey. *Annals of Discrete Mathematics*, 5:287–326, 1979.
- [13] R. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher, editors, *Complexity of computer computations (Proc. Sympos., IBM Thomas J. Watson Res. Center, Yorktown Heights, N.Y., 1972)*, pages 85–103. Plenum, New York, 1972.

- [14] J. Labetoulle, E. Lawler, J. Lenstra, and A. Rinnooy Kan. Preemptive scheduling of uniform machines subject to release dates. In *Progress in combinatorial optimization (Waterloo, Ont., 1982)*, pages 245–261. Academic Press, Toronto, Ont., 1984.
- [15] E. Lawler. Optimal sequencing of a single machine subject to precedence constraints. *Management Science*, 19:544–546, 1973.
- [16] E. Lawler. A “pseudopolynomial” algorithm for sequencing jobs to minimize total tardiness. *Annals of Discrete Mathematics*, 1:331–342, 1977.
- [17] E. Lawler. Sequencing jobs to minimize total weighted completion time subject to precedence constraints. *Annals of Discrete Mathematics*, 2:75–90, 1978.
- [18] E. Lawler. A dynamic programming algorithm for preemptive scheduling of a single machine to minimize the number of late jobs. *Annals of Operations Research*, 26(1-4):125–133, 1990.
- [19] E. Lawler and J. Moore. A functional equation and its application to resource allocation and sequencing problems. *Management Science*, 16:77–84, 1969.
- [20] J. Lenstra. Not published.
- [21] J. Lenstra and A. Rinnooy Kan. Complexity of scheduling under precedence constraints. *Operations Research*, 26(1):22–35, 1978.
- [22] J. Lenstra and A. Rinnooy Kan. Complexity results for scheduling chains on a single machine. *European Journal of Operational Research*, 4(4):270–275, 1980.
- [23] J. Lenstra, A. Rinnooy Kan, and P. Brucker. Complexity of machine scheduling problems. *Annals of Discrete Mathematics*, 1:343–362, 1977.
- [24] J.-T. Leung and G. Young. Minimizing total tardiness on a single machine with precedence constraints. *ORSA Journal on Computing*, 2(4):346–352, 1990.
- [25] B. Simons. A fast algorithm for single processor scheduling. In *19th Annual Symposium on Foundations of Computer Science (Ann Arbor, Mich., 1978)*, pages 246–252. IEEE, Long Beach, Calif., 1978.
- [26] B. Simons. Multiprocessor scheduling of unit-time jobs with arbitrary release times and deadlines. *SIAM Journal on Computing*, 12(2):294–299, 1983.
- [27] W. Smith. Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3:59–66, 1956.
- [28] Z. Tian, C. Ng, and T. Cheng. An $O(n^2)$ algorithm for scheduling equal-length preemptive jobs on a single machine to minimize total tardiness. *Journal of Scheduling*, 9(4):343–364, 2006.

References

Preface

[1] J. Blazewicz, K. H. Ecker, E. Pesch, G. Schmidt, and J. Weglarz, editors. Handbook on Scheduling: From Theory to Applications. Springer, 2007.

[2] P. Chre'tienne, E. G. Coffman Jr., J. K. Lenstra, and Z. Liu, editors. Scheduling Theory and its Applications. John Wiley & Sons, 1995.

[3] J. Y.-T. Leung, editor. Handbook of Scheduling: Algorithms, Models, and Performance Analysis. Chapman & Hall/CRC, 2004.

[4] B. A. Shirazi, A. R. Hurson, and K. M. Kavi. Scheduling and load balancing in parallel and distributed systems. IEEE Computer Science Press, 1995.

1 Chapter 1: On the Complexity of Scheduling

[1] K. Baker. Introduction to Sequencing and Scheduling. John Wiley & Sons, New York, 1974.

[2] K. Baker, E. Lawler, J. Lenstra, and A. Rinnooy Kan. Preemptive scheduling of a single machine to minimize maximum cost subject to release dates and precedence constraints. *Operations Research*, 31:381- 386, 1983.

[3] P. Baptiste. Polynomial time algorithms for minimizing the weighted number of late jobs on a single machine with equal processing times. *Journal of Scheduling*, 2:245-252, 1999.

[4] P. Baptiste. Scheduling equal-length jobs on identical parallel machines. *Discrete Applied Mathematics*, 103(1):21-32, 2000.

[5] P. Baptiste, P. Brucker, S. Knust, and V. Timkovsky. Ten notes on equal-execution-time scheduling. *4OR*, 2:111-127, 2004.

[6] J. Błazewicz. Scheduling dependent tasks with different arrival times to meet deadlines. In *Model. Perform. Eval. Comput. Syst., Proc. int. Workshop, Stresa 1976*, pages 57-65. North Holland, Amsterdam, 1976.

[7] J. Błazewicz, K. Ecker, E. Pesch, G. Schmidt, and J. Weglarz. *Scheduling Computer and Manufacturing Processes*. Springer, Berlin, 2nd edition, 2001.

[8] P. Brucker. *Scheduling Algorithms*. Springer, Berlin, 5th edition, 2007.

[9] P. Brucker and S. Knust. *Complex Scheduling*. Springer, Berlin, 2006.

[10] J. Du and J.-T. Leung. Minimizing total tardiness on one machine is NP-hard. *Mathematics of Operations Research*, 15(3):483-495, 1990.

[11] M. R. Garey and D. S. Johnson. *Computers and Intractability. A Guide to the Theory of NP-Completeness*. W. H. Freeman & Co, San Francisco, 1979.

[12] R. Graham, E. Lawler, J. Lenstra, and A. Rinnooy Kan. Optimization and approximation in deterministic sequencing and scheduling: A survey. *Annals of Discrete Mathematics*,

5:287-326, 1979.

[13] R. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher, editors, Complexity of computer computations (Proc. Sympos., IBM Thomas J. Watson Res. Center, Yorktown Heights, N.Y., 1972), pages 85-103. Plenum, New York, 1972. Rinnooy Kan. scheduling of uniform machines subject to release dates. In Progress in combinatorial optimization (Waterloo, Ont., 1982), pages 245-261. Academic Press, Toronto, Ont., 1984.

[15] E. Lawler. Optimal sequencing of a single machine subject to precedence constraints. Management Science, 19:544-546, 1973.

[16] E. Lawler. A "pseudopolynomial" algorithm for sequencing jobs to minimize total tardiness. Annals of Discrete Mathematics, 1:331-342, 1977.

[17] E. Lawler. Sequencing jobs to minimize total weighted completion time subject to precedence constraints. Annals of Discrete Mathematics, 2:75-90, 1978.

[18] E. Lawler. A dynamic programming algorithm for preemptive scheduling of a single machine to minimize the number of late jobs. Annals of Operations Research, 26(1-4):125-133, 1990.

[19] E. Lawler and J. Moore. A functional equation and its application to resource allocation and sequencing problems. Management Science, 16:77- 84, 1969.

[20] J. Lenstra. Not published.

[21] J. Lenstra and A. Rinnooy Kan. Complexity of scheduling under precedence constraints. Operations Research, 26(1):22-35, 1978.

[22] J. Lenstra and A. Rinnooy Kan. Complexity results for scheduling chains on a single machine. European Journal of Operational Research, 4(4):270-275, 1980.

[23] J. Lenstra, A. Rinnooy Kan, and P. Brucker. Complexity of machine scheduling problems. Annals of Discrete Mathematics, 1:343-362, 1977.

[24] J.-T. Leung and G. Young. Minimizing total tardiness on a single machine with precedence constraints. ORSA Journal on Computing, 2(4):346-352, 1990.

- [25] B. Simons. A fast algorithm for single processor scheduling. In 19th Annual Symposium on Foundations of Computer Science (Ann Arbor, Mich., 1978), pages 246-252. IEEE, Long Beach, Calif., 1978.
- [26] B. Simons. Multiprocessor scheduling of unit-time jobs with arbitrary release times and deadlines. SIAM Journal on Computing, 12(2):294- 299, 1983.
- [27] W. Smith. Various optimizers for single-stage production. Naval Research Logistics Quarterly, 3:59-66, 1956.
- [28] Z. Tian, C. Ng, and T. Cheng. An $O(n^2)$ algorithm for scheduling equal length preemptive jobs on a single machine to minimize total tardiness. Journal of Scheduling, 9(4):343-364, 2006.

2 Chapter 2: Approximation Algorithms for Scheduling Problems

- [1] G. Ausiello, P. Crescenzi, G. Gambosi, V. Kann, A. MarchettiSpaccamela, and M. Protasi. Complexity and Approximation: Combinatorial Optimization Problems and Their Approximability Properties, chapter 3, pages 100–102. Springer-Verlag, 1999.
- [2] E. Bampis, A. Giannakos, and J.-C. Ko“nig. On the complexity of scheduling with large communication delays. *European Journal of Operational Research*, 94(2):252–260, 1996.
- [3] J. Błazewicz, K. Ecker, E. Pesch, G. Schmidt, and J. Weglarz. *Handbook on Scheduling: From Theory to Applications*. International Handbooks on Information Systems. Springer, 2007.
- [4] R. Brent. The parallel evaluation of general arithmetic expression. *Journal of the ACM*, 21(2):201–206, 1974.
- [5] H. Casanova, A. Legrand, and Y. Robert. *Parallel Algorithms*. Chapman & Hall/CRC Press, 2008.
- [6] P. Chre’tienne and J. Colin. C.P.M. scheduling with small interprocessor communication delays. *Operations Research*, 39(3):680–684, 1991.
- [7] P. Chre’tienne and C. Picouleau. Scheduling with communication delays: A survey. In P. Chre’tienne, E. C. Jr., J. Lenstra, and Z. Liu, editors, *Scheduling Theory and Its Applications*. John Wiley & Sons, 1995.
- [8] M. R. Garey and D. S. Johnson. *Computers and Intractability. A Guide to the Theory of NP-Completeness*. W. H. Freeman & Co, San Francisco, 1979.
- [9] R. Giroudeau, J.-C. Ko“nig, F. K. Moulai, and J. Palaysi. Complexity and approximation for precedence constrained scheduling problems with large communication delays. *Theoretical Computer Science*, 401(13):107–119, 2008.
- [10] R. Graham, E. Lawler, J. Lenstra, and A. Rinnooy Kan. Optimization and approximation in deterministic sequencing and scheduling: A survey. *Annals of Discrete Mathematics*, 5:287–326, 1979.
- [11] I. Holyer. The NP-completeness of edge-coloring. *SIAM*

Journal on Computing, 10:718-720, 1981.

[12] A. Munier and C. Hanen. An approximation algorithm for scheduling unitary tasks on m processors with communication delays. Private communication, 1996. for scheduling unitary on m processors with unit communication delays. Theoretical Computer Science, 178(1-2):119-127, 1997.

[14] A. Munier and J. Ko"nig. A heuristic for a scheduling problem with communication delays. Operations Research, 45(1):145-148, 1997.

[15] C. Papadimitriou and M. Yannakakis. Towards an architectureindependent analysis of parallel algorithms. SIAM Journal on Computing, 19(2):322-328, 1990.

[16] C. Picouleau. New complexity results on scheduling with small communication delays. Discrete Applied Mathematics, 60:331-342, 1995.

[17] C. Rapine. Algorithmes d'approximation garantie pour l'ordonnancement de ta`ches, Application au domaine du calcul paralle`le. The`se de doctorat, Institut National Polytechnique de Grenoble, 1999.

[18] R. Saad. Scheduling with communication delays. Journal of Combinatorial Mathematics and Combinatorial Computing, 18:214-224, 1995.

[19] R. Thurimella and Y. Yesha. A scheduling principle for precedence graphs with communication delay. In International Conference on Parallel Processing, volume 3, pages 229-236, 1992.

[20] V. V. Vazirani. Approximation Algorithms. Springer, 2nd edition, 2002.

[21] B. Veltman. Multiprocessor Scheduling with Communications Delays. Ph.D. thesis, CWI-Amsterdam, 1993.

[22] V. Vizing. On an estimate of the chromatic class of a p -graph. Diskret Analiz, 3:23-30, 1964.

3 Chapter 3: Online Scheduling

- [1] S. Albers. Better bounds for online scheduling. *SIAM Journal on Computing*, 29:459-473, 1999.
- [2] S. Albers. On randomized online scheduling. In 34th ACM Symposium on Theory of Computing, pages 134-143, 2002.
- [3] S. Albers and H. Fujiwara. Energy-efficient algorithms for flow time on Theoretical of Computer Science (STACS), volume 3884 of LNCS, pages 621-633. Springer, 2006.
- [4] S. Albers, F. Müller, and S. Schmelzer. Speed scaling on parallel processors. In 19th ACM Symposium on Parallelism in Algorithms and Architectures, pages 289-298, 2007.
- [5] A. Armon, Y. Azar, and L. Epstein. Temporary tasks assignment resolved. *Algorithmica*, 36(3):295-314, 2003.
- [6] J. Augustine, S. Irani, and C. Swamy. Optimal power-down strategies. *SIAM Journal on Computing*, 37(5):1499-1516, 2008.
- [7] B. Awerbuch, Y. Azar, S. Leonardi, and O. Regev. Minimizing the flow time without migration. *SIAM Journal on Computing*, 31(5):1370-1382, 2002.
- [8] Y. Azar, A. Broder, and A. Karlin. On-line load balancing. *Theoretical Computer Science*, 130(1):73-84, 1994.
- [9] Y. Azar, A. Epstein, and L. Epstein. Load balancing of temporary tasks in the l_p norm. *Theoretical Computer Science*, 361(2-3):314-328, 2006.
- [10] Y. Azar and L. Epstein. On-line load balancing of temporary tasks on identical machines. *SIAM Journal on Discrete Mathematics*, 18(2):347- 352, 2004.
- [11] Y. Azar, B. Kalyanasundaram, S. Plotkin, K. Pruhs, and O. Waarts. Online load balancing of temporary tasks. *Journal of Algorithms*, 22(1):93- 110, 1997.
- [12] K. Baker. *Introduction to Sequencing and Scheduling*. John Wiley & Sons, New York, 1974.
- [13] N. Bansal, T. Kimbrel, and K. Pruhs. Dynamic speed scaling to manage energy and temperature. *Journal of the ACM*, 54(1), 2007.

- [14] N. Bansal, K. Pruhs, and C. Stein. Speed scaling for weighted flow time. In 18th Annual ACM-SIAM Symposium on Discrete Algorithms, pages 805-813, 2007.
- [15] L. Barroso. The price of performance. ACM Queue, 3(7):48-53, Sept. 2005.
- [16] Y. Bartal, A. Fiat, H. Karloff, and R. Vohra. New algorithms for an ancient scheduling problem. J. of Computer and System Sciences, 51:359- 366, 1995.
- [17] L. Becchetti and S. Leonardi. Nonclairvoyant scheduling to minimize the total flow time on single and parallel machines. Journal of the ACM, 51(4):517-539, 2004. and flow. In 18th nual ACM Symposium on Parallel Algorithms and Architectures, pages 190-196, 2006.
- [19] H.-L. Chan, W.-T. Chan, T.-W. Lam, L.-K. Lee, K.-S. Mak, and P. Wong. Energy efficient online deadline scheduling. In 18th Annual ACM-SIAM Symposium on Discrete Algorithms, pages 795-804, 2007.
- [20] C. Chekuri, S. Khanna, and A. Zhu. Algorithms for minimizing weighted flow time. In 33rd Annual ACM Symposium on Theory of Computing, pages 84-93, 2001.
- [21] B. Chen, A. van Vliet, and G. Woeginger. A lower bound for randomized on-line scheduling algorithms. Information Processing Letters, 51:219- 222, 1994.
- [22] U. Faigle, W. Kern, and G. Turan. On the performance of on-line algorithms for particular problems. Acta Cybernetica, 9:107-119, 1989.
- [23] R. Fleischer and M. Wahl. Online scheduling revisited. Journal of Scheduling, 3:343-353, 2000.
- [24] G. Galambos and G. Woeginger. An on-line scheduling heuristic with better worst case ratio than Graham's list scheduling. SIAM Journal on Computing, 22:349-355, 1993.
- [25] B. Gaujal and N. Navet. Dynamic voltage scaling under edf revisited. Real-Time Systems, 37(1):77-97, 2007.
- [26] B. Gaujal, N. Navet, and C. Walsh. Shortest path algorithms for realtime scheduling of fifo tasks with optimal energy use. ACM Transactions on Embedded Computing Systems, 4(4):907-933, 2005.

- [27] R. L. Graham. Bounds for certain multiprocessor anomalies. *Bell System Technical Journal*, 45:1563-1581, 1966.
- [28] S. Irani, R. Gupta, and S. Shukla. Competitive analysis of dynamic power management strategies for systems with multiple power savings states. In *Design, Automation and Test in Europe, Conference and Exposition*, pages 117-123, 2002.
- [29] S. Irani and A. Karlin. Online computation. In D. Hochbaum, editor, *Approximation Algorithms for NP-Hard Problems*, pages 521-564. PWS Publishing Company, 1997.
- [30] S. Irani and K. Pruhs. Algorithmic problems in power management. *ACM SIGACT News*, 36(2):63-76, 2005.
- [31] B. Kalyanasundaram and K. Pruhs. Speed is as powerful as clairvoyance. *Journal of the ACM*, 47(4):617-643, 2000.
- [32] J. L. Lawler. A note on the complexity of the flow time antly. *Journal of the ACM*, 50(4):551-567, 2003.
- [33] D. Karger, S. Phillips, and E. Torng. A better algorithm for an ancient scheduling problem. *Journal of Algorithms*, 20:400-430, 1996.
- [34] A. Karlin, M. Manasse, L. McGeoch, and S. Owicki. Competitive randomized algorithms for nonuniform problems. *Algorithmica*, 11(6):542- 571, 1994.
- [35] A. Legrand, A. Su, and F. Vivien. Minimizing the stretch when scheduling flows of divisible requests. *Journal of Scheduling*, 11(5):381-404, 2008.
- [36] S. Leonardi. A simpler proof of preemptive total flow time approximation on parallel machines. In *Efficient Approximation and Online Algorithms*, volume 3484 of LNCS, pages 203-212. Springer, 2003.
- [37] S. Leonardi and D. Raz. Approximating total flow time on parallel machines. In *29th ACM Symposium on Theory of Computing*, pages 110-119, 1997.
- [38] S. Leonardi and D. Raz. Approximating total flow time on parallel machines. *J. of Computer and System Sciences*, 73(6):875-891, 2007.
- [39] M. Li and F. Yao. An efficient algorithm for computing optimal discrete voltage schedules. *SIAM Journal on Computing*, 35(3):658-671, 2005.

- [40] J. McCullough and E. Torng. Srpt optimally utilizes faster machines to minimize flow time. In 15th Annual ACM-SIAM Symposium on Discrete Algorithms, pages 350-358, 2004.
- [41] R. Motwani, S. Phillips, and E. Torng. Non-clairvoyant scheduling. Theoretical Computer Science, 130:17-47, 1994.
- [42] S. Muthukrishnan, R. Rajaraman, A. Shaheen, and J. Gehrke. Online scheduling to minimize average stretch. SIAM Journal on Computing, 34(2):433-452, 2004.
- [43] K. Pruhs, J. Sgall, and E. Torng. Online scheduling. In J. Leung, editor, Handbook of Scheduling, chapter 15. Chapman & Hall/CRC, 2004.
- [44] K. Pruhs, P. Uthaisombut, and G. Woeginger. Getting the best response for your erg. In 9th Scandinavian Workshop on Algorithm Theory (SWAT), volume 3111 of LNCS, pages 14-25. Springer, 2004.
- [45] J. Rudin III. Improved bounds for the on-line scheduling problem. PhD thesis, The University of Texas at Dallas, May 2001.
- [46] S. Seiden. Online randomized multiprocessor scheduling. Algorithmica, 28:73-216, 2000. multiprocessor ing. Information Processing Letters, 63:51-55, 1997.
- [48] D. Sleator and R. Tarjan. Amortized efficiency of list update and paging rules. Communications of the ACM, 28:202-208, 1985.
- [49] F. Yao, A. Demers, and S. Shenker. A scheduling model for reduced cpu energy. In 36th Annual Symposium on Foundations of Computer Science, pages 374-382, 1995.

4 Chapter 4: Job Scheduling

[1] F. Afrati, E. Bampis, C. Chekuri, D. Karger, C. Kenyon, S. Khanna, I. Milis, M. Queyranne, M. Skutella, C. Stein, and M. Sviridenko. Approximation schemes for minimizing average weighted completion time with release dates. In Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS), pages 32-44, 1999.

[2] E. Anderson and C. Potts. Online scheduling of a single machine to minimize total weighted completion time. *Mathematics of Operations Research*, 29(3):686-697, 2004.

[3] C. Chekuri, R. Motwani, B. Natarajan, and C. Stein. Approximation techniques for average completion time scheduling. *SIAM Journal on Computing*, 31(1):146-166, 2001.

[4] B. Chen and A. Vestjens. Scheduling on identical machines: How good is LPT in an on-line setting? *Operations Research Letters*, 21(4):165-169, 1997.

[5] R. Conway, W. Maxwell, and L. Miller. *Theory of Scheduling*. AddisonWesley, 1967.

[6] A. Fishkin, K. Jansen, and L. Porkolab. On minimizing average weighted completion time of multiprocessor tasks with release dates. In Proceedings of 28th International Colloquium on Automata, Languages and Programming ICALP 2001, volume 2076 of Lecture Notes in Computer Science, pages 875-886. Springer-Verlag, 2001.

[7] M. Garey and R. Graham. Bounds for multiprocessor scheduling with resource constraints. *SIAM Journal on Computing*, 4(2):187-200, June 1975.

[8] M. Goemans, M. Queyranne, A. Schulz, M. Skutella, and Y. Wang. Single machine scheduling with release dates. *SIAM Journal on Discrete Mathematics*, 15:165-192, 2002.

[9] R. Graham. Bounds on multiprocessor timing anomalies. *SIAM Journal on Applied Mathematics*, 17:416-429, 1969.

[10] R. L. Graham. Bounds for certain multiprocessor anomalies. *Bell System Technical Journal*, 45:1563-1581, 1966.

[11] L. Hall and D. Shmoys. Approximation schemes for constrained scheduling problems. In Proceedings of the 30th Annual Symposium on Foundations of Computer Science, pages 134-139, 1989. algorithms for machine scheduling. In 5th

International Integer Programming and Combinatorial Optimization Conference, volume 1084 of Lecture Notes in Computer Science, pages 404-414. Springer-Verlag, 1996.

[13] M. Hussein and U. Schwiegelshohn. Utilization of nonclairvoyant online schedules. *Theoretical Computer Science*, 362:238-247, 2006.

[14] T. Kawaguchi and S. Kyan. Worst case bound of an LRF schedule for the mean weighted flow-time problem. *SIAM Journal on Computing*, 15(4):1119-1129, Nov. 1986.

[15] H. Kellerer, T. Tautenhahn, and G. Wo"ginger. Approximability and nonapproximability results for minimizing total flow time on a single machine. *SIAM Journal on Computing*, 28(4):1155-1166, 1999.

[16] R. McNaughton. Scheduling with deadlines and loss functions. *Management Science*, 6(1):1-12, Oct. 1959.

[17] N. Megow and A. Schulz. On-line scheduling to minimize average completion time revisited. *Operations Research Letters*, 32(5):485-490, 2004.

[18] R. Motwani, S. Phillips, and E. Torng. Non-clairvoyant scheduling. *Theoretical Computer Science*, 130:17-47, 1994.

[19] G. Mounie', C. Rapine, and D. Trystram. A $3/2$ approximation algorithm for scheduling independent monotonic malleable tasks. *SIAM Journal on Computing*, 37(2):401-412, 2007.

[20] E. Naroska and U. Schwiegelshohn. On an online scheduling problem for parallel jobs. *Information Processing Letters*, 81(6):297-304, Mar. 2002.

[21] C. Phillips, C. Stein, and J. Wein. Minimizing average completion time in the presence of release dates. *Mathematical Programming*, 82:199-223, 1998.

[22] M. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Prentice-Hall, New Jersey, second edition, 2002.

[23] M. Queyranne. Structure of a simple scheduling polyhedron. *Mathematical Programming*, 58(2):263-285, 1993.

[24] A. S. Schulz and M. Skutella. The power of α -points in preemptive single machine scheduling. *Journal of Scheduling*, 5:121-133, 2002.

- [25] A. S. Schulz and M. Skutella. Scheduling unrelated machines by randomized rounding. *SIAM Journal on Discrete Mathematics*, 15(4):450-469, 2002.
- [26] U. Schwiegelshohn. Preemptive weighted completion time scheduling of parallel jobs. *SIAM Journal on Computing*, 33(6):1280-1308, 2004. Turek, and P. Yu. SMART bounds for weighted response time scheduling. *SIAM Journal on Computing*, 28(1):237-253, Jan. 1999.
- [28] U. Schwiegelshohn and R. Yahyapour. Fairness in parallel job scheduling. *Journal of Scheduling*, 3(5):297-320, 2000.
- [29] D. B. Shmoys, J. Wein, and D. P. Williamson. Scheduling parallel machines on-line. *SIAM Journal on Computing*, 24(6):1313-1331, Dec. 1995.
- [30] W. Smith. Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3:59-66, 1956.

5 Chapter 5: Cyclic Scheduling

- [1] D. Alacaide, C. Chu, V. Kats, E. Levner, and G. Sierksma. Cyclic multiple robot scheduling with time-window constraints using a critical path approach. *European Journal of Operational Research*, 177:147-162, 2007.
- [2] F. Bacelli, G. Cohen, G. J. Olsder, and J.-P. Quadrat. *Synchronisation and Linearity: An Algebra for Discrete Event Systems*. John Wiley & Sons, New York, 1992.
- [3] P. Brucker and T. Kampmeyer. Tabu search algorithms for cyclic machine scheduling problems. *Journal of Scheduling*, 8(4):303-322, 2005.
- [4] K. Caggiano and P. Jackson. Finding minimum flow time cyclic schedules for non identical, multistage jobs. *IIE Transactions*, 40:45-65, 2008.
- [5] P.-Y. Calland, A. Darte, and Y. Robert. Circuit retiming applied to decomposed software pipelining. *IEEE Transactions on Parallel and Distributed Systems*, 9(1):24-35, 1998.
- [6] J. Carlier and P. Chre'tienne. *Proble'mes D'ordonnancement: Mode'lisat'ion, Complexite', Algorithmes*. Masson, Paris, 1988.
- [7] J. Carlier and P. Chre'tienne. Timed Petri nets schedules. In *Advances in Petri nets*, volume 340 of *Lectures Notes in Computer Sciences*, pages 62-84. Springer-Verlag, 1988.
- [8] P. Chre'tienne. Transient and limiting behavior of timed event graphs. *RAIRO Techniques et Sciences Informatiques*, 4:127-192, 1985.
- [9] P. Chre'tienne. List schedules for cyclic scheduling. *Discrete Applied Mathematics*, 94:141-159, 1999.
- [10] P. Chre'tienne. On Graham's bound for cyclic scheduling. *Parallel Computing*, 26(9):1163-1174, 2000.
- [11] G. Cohen, P. Moller, J.-P. Quadrat, and M. Viot. Algebraic tools for the performance evaluation of discrete event systems. *IEEE Proceeding: Special Issue on Discrete Event Dynamics Systems*, 77(1):39-58, 1989.
- [12] T. Cormen, C. Leiserson, and R. Rivest. *Introduction to Algorithms*. MIT Press, 1990.

- [13] A. Darte and G. Huard. Loop shifting for loop compaction. *International Journal of Parallel Programming*, 28(5):499-534, 2000. K. algorithms for mum cycle mean and optimum cost to time ratio problems. In *Design Automation Conference*, pages 37-42, 1999.
- [15] D. de Werra, C. Eisenbeis, S. Lelait, and B. Marmol. On a graphtheoretical model for cyclic register allocation. *Discrete Applied Mathematics*, 93:191-203, 1999.
- [16] B. Dupont de Dinechin, C. Artigues, and S. Azem. Resource constrained modulo scheduling. In C. Artigues, S. Demasse, and E. Neron, editors, *Resource-Constrained Project Scheduling*, pages 267-277. ISTE and John Wiley, London, 2008.
- [17] A. Eichenberger and E. Davidson. Efficient formulation for optimal modulo schedulers. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 194-205, Las Vegas, Nevada, 1997.
- [18] F. Gasperoni and U. Schwiegelshohn. Generating close to optimum loop schedules on parallel processors. *Parallel Processing Letters*, 4:391-403, 1994.
- [19] M. Gondran and M. Minoux. *Graphs and Algorithms*. John Wiley & Sons, first edition, 1984.
- [20] N. G. Hall, T.-E. Lee, and T.-E. Posner. The complexity of cyclic scheduling problems. *Journal of Scheduling*, 5:307-327, 2002.
- [21] C. Hanen. Study of an NP-hard cyclic scheduling problem: the recurrent job-shop. *European Journal of Operational Research*, 72(1):82-101, 1994.
- [22] C. Hanen and A. Munier. Cyclic scheduling on parallel processors: An overview. In P. Chre'tienne, E. G. Coffman, J. K. Lenstra, and Z. Liu, editors, *Scheduling Theory and Its Applications*. John Wiley & Sons, 1994.
- [23] C. Hanen and A. Munier. A study of the cyclic scheduling problem on parallel processors. *Discrete Applied Mathematics*, 57(2-3):167-192, 1995.
- [24] M. Hartmann and C. Arguelles. Transience bounds for long walks. *Mathematics of Operation Research*, 24(2), 1999.
- [25] H. Hillion and J.-M. Proth. Performance evaluation of

job-shop systems using timed event graph. IEEE Transactions on Automatic Control, 34(1):3-9, 1989.

[26] E. Levner and V. Kats. A parametric critical path problem and an application for cyclic scheduling. Discrete Applied Mathematics, 87:149- 158, 1998. for periodic of tasks on parallel processors. In L. Yang and M. Paprzycki, editors, Practical Applications of Parallel Computing: Advances in Computation Theory and Practice, volume 12, pages 363-370. Nova Science Publishers, Canada, 2003.

[28] E. Levner, V. Kats, and D. Alcaide. Cyclic scheduling in robotic cells: An extension of basic models in machine scheduling theory. In E. Levner, editor, Multiprocessor Scheduling: Theory and Applications, pages 1-20. I-Tech Education and Publishing, Vienna, Austria, 2007.

[29] T. McCormick and U. S. Rao. Some complexity results in cyclic scheduling. Mathematical and Computer Modelling, 20:107-122, 1994.

[30] A. Munier. Contribution a` l'e´tude des ordonnancements cycliques. Ph.D. thesis, 1991.

[31] A. Munier-Kordon. A generalization of the basic cyclic scheduling problem. Journal of Scheduling, 2009.

[32] R. Roundy. Cyclic schedules for job-shop with identical jobs. Mathematics of Operation Research, 17(4):842-865, 1992.

[33] P. Sucha, Z. Pohl, and Z. Hanzalek. Scheduling of iterative algorithms on FPGA with pipelined arithmetic unit. In 10th IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS, 2004.

[34] S. Touati. On the periodic register need in software pipelining. IEEE Transactions on Computers, 56(11):1493-1504, 2007.

[35] B. Trouillet, O. Korbaa, and J.-C. Gentina. Formal approach of FMS cyclic scheduling. IEEE Transactions on Systems, Man and Cybernetics. Part C: Applications and Reviews, 37(1):126-137, 2007.

6 Chapter 6: Cyclic Scheduling for the Synthesis of Embedded Systems

- [1] M. Ade'. Data Memory Minimization for Synchronous Data Flow Graphs Emulated on DSP-FPGA Targets. Ph.D. thesis, Universite' Catholique de Louvain, 1997.
- [2] M. Ade', R. Lauwereins, and J. A. Peperstraete. Buffer memory requirements in DSP applications. In IEEE 5th International Workshop on Rapid System Prototyping, pages 108-123, Grenoble, France, 1994.
- [3] A. Benabid, C. Hanen, O. Marchetti, and A. Munier-Kordon. Periodic schedules for unitary timed weighted event graphs. In ROADEF 2008 conference, Clermont-Ferrand, France, 2008. A. Lee. Software from Dataflow Graphs. Kluwer, first edition, 1996.
- [5] S. S. Bhattacharyya, P. K. Murthy, and E. A. Lee. Synthesis of embedded software from synchronous dataflow specifications. Journal of VLSI Signal Processing, 21(2):151-166, 1999.
- [6] P. Chrzastowski-Wachtel and M. Racunas. Liveness of weighted circuits and the diophantine problem of Frobenius. In Proceeding of FCT'93, volume 710 of Lecture Notes in Computer Science, pages 171-180. Springer, 1993.
- [7] T. Cormen, C. Leiserson, and R. Rivest. Introduction to Algorithms. MIT Press, 1990.
- [8] M. C'ubric' and P. Panangaden. Minimal memory schedules for dataflow networks. In CONCUR '93, 4th International Conference on Concurrency Theory, volume 715 of Lecture Notes in Computer Science, pages 368-383, 1993.
- [9] M. R. Garey and D. S. Johnson. Computers and Intractability. A Guide to the Theory of NP-Completeness. W. H. Freeman & Co, San Francisco, 1979.
- [10] S. Gaubert. An algebraic method for optimizing resources in timed event graphs. In Proceedings of the 9th conference on Analysis and Optimization of Systems, volume 144 of Lecture Notes in Computer Science, pages 957-966. Springer-Verlag, 1990.
- [11] M. Geilen, T. Basten, and S. Stuijk. Minimising buffer requirements of synchronous dataflow graphs with model cheking. In Proocedings of the DAC, 2005.

- [12] A. Ghamarian, M. Geilen, T. Basten, A. Moonen, M. Bekooij, B. Theelen, and M. Mousavi. Throughput analysis of synchronous data flow graphs. In Proceedings of the Sixth IEEE International Conference on Applications of Concurrency for System Design, 2006.
- [13] A. Giua, A. Piccaluga, and C. Seatzu. Firing rate optimization of cyclic timed event graphs. *Automatica*, 38(1):91-103, Jan. 2002.
- [14] H. Hillion and J. Proth. Performance evaluation of a job-shop system using timed event graph. *IEEE Transactions on Automatic Control*, 34(1):3-9, 1989.
- [15] M. Karczmarek. Constrained and Phased Scheduling of Synchronous Data Flow Graphs for StreamIt Language. Master's thesis, Massachusetts Institute of Technology, 2002.
- [16] R. M. Karp. A characterization of the minimum cycle mean in a digraph. *Discrete Mathematics*, 23, 1978. a model for parallel tations: Determinacy, termination, queueing. *SIAM Journal on Applied Mathematics*, 14(6):1390-1411, 1966.
- [18] S. Lafit, J. Proth, and X. Xie. Optimization of invariant criteria for event graphs. *IEEE Transactions on Automatic Control*, 37(5):547-555, 1992.
- [19] E. A. Lee. Overview of the Ptolemy project. Technical report, University of California, Berkeley, 2003.
- [20] E. A. Lee and D. G. Messerschmitt. Static scheduling of synchronous data flow programs for digital signal processings. *IEEE Transaction on Computers*, 36(1):24-35, 1987.
- [21] E. A. Lee and D. G. Messerschmitt. Synchronous data flow. *Proceedings of the IEEE*, 75(9):1235-1245, 1987.
- [22] O. Marchetti and A. Munier-Kordon. Complexity results for bi-criteria cyclic scheduling problems. In 17e`mes Rencontres Francophones du Parallélisme (RenPar'2006), Perpignan, France, pages 17-24, 2006.
- [23] O. Marchetti and A. Munier-Kordon. Minimizing place capacities of weighted event graphs for enforcing liveness. *Discrete Event Dynamic Systems*, 18(1):91-109, 2008.
- [24] O. Marchetti and A. Munier-Kordon. A sufficient condition for the liveness of weighted event graphs.

European Journal of Operational Research, 197(2):532-540, 2009.

[25] A. Munier. Re'gime asymptotique optimal d'un graphe d'e've'nements tempore's ge'ne'ralise': application a` un proble'me d'assemblage. *RAIROAutomatique Productique Informatique Industrielle*, 27(5):487-513, 1993.

[26] A. Munier. The basic cyclic scheduling problem with linear precedence constraints. *Discrete Applied Mathematics*, 64(3):219-238, 1996.

[27] P. K. Murthy, S. S. Bhattacharyya, and E. A. Lee. Joint minimization of code and data for synchronous dataflow programs. *Formal Methods in System Design*, 11(1):41-70, 1997.

[28] P. M. Murthy. Scheduling Techniques for Synchronous and Multidimensional Synchronous Dataflow. Ph.D. thesis, University of California at Berkeley, 1996.

[29] J.-L. Peterson. *Petri Net Theory and the Modeling of Systems*. Prentice Hall PTR, 1981.

[30] N. Sauer. Marking optimization of weighted marked graph. *Discrete Event Dynamic Systems*, 13(3):245-262, 2003. trade-offs in quirements and throughput constraints for synchronous dataflow graphs. In *DAC '06: Proceedings of the 43rd Annual Conference on Design automation*, pages 899-904, New York, NY, USA, 2006. ACM.

[32] E. Teruel, P. Chrzastowski-Wachtel, J. M. Colom, and M. Silva. On weighted T-systems. In *Proceedings of the 13th International Conference on Application and Theory of Petri Nets 1992*, volume 616 of *Lecture Notes in Computer Science*. Springer, 1992.

[33] W. Thies, M. Karczmarek, M. Gordon, D. Z. Maze, J. Wong, H. Hoffman, M. Brown, and S. Amarasinghe. *StreamIt: A compiler for streaming applications*. Technical report, Massachusetts Institute of Technology, 2001.

7 Chapter 7: Steady-State Scheduling

[1] B. Awerbuch and T. Leighton. Improved approximation algorithms for the multi-commodity flow problem and local competitive routing in dynamic networks. In 26-th Annual ACM Symposium on Theory of Computing, pages 487-496, 1994.

[2] O. Beaumont. Nouvelles me´thodes pour l'ordonnancement sur platesformes he´te´roge`nes. Habilitation a` diriger des recherches, Universite´ de Bordeaux 1, Dec. 2004.

[3] O. Beaumont, A. Legrand, L. Marchal, and Y. Robert. Assessing the impact and limits of steady-state scheduling for mixed task and data parallelism on heterogeneous platforms. Research report 2004-20, LIP, ENS Lyon, France, Apr. 2004. Also available as INRIA Research Report RR-5198.

[4] O. Beaumont, A. Legrand, L. Marchal, and Y. Robert. Complexity results and heuristics for pipelined multicast operations on heterogeneous platforms. In Proceedings of the 33rd International Conference on Parallel Processing (ICPP). IEEE Computer Society Press, 2004.

[5] O. Beaumont, A. Legrand, L. Marchal, and Y. Robert. Pipelining broadcasts on heterogeneous platforms. IEEE Transactions on Parallel Distributed Systems, 16(4):300-313, 2005.

[6] O. Beaumont, A. Legrand, L. Marchal, and Y. Robert. Steady-state scheduling on heterogeneous clusters. International Journal of Foundations of Computer Science, 16(2):163-194, 2005.

[7] D. Bertsimas and D. Gamarnik. Asymptotically optimal algorithm for job shop scheduling and packet routing. Journal of Algorithms, 33(2):296-318, 1999.

[8] Berkeley Open Infrastructure for Network Computing. <http://boinc.berkeley.edu>.

[9] H. Casanova and F. Berman. Grid Computing: Making The Global Infrastructure a Reality, chapter Parameter Sweeps on the Grid with APST. John Wiley, 2003. Hey, A. and Berman, F. and Fox, G., editors.

[10] T. Cormen, C. Leiserson, and R. Rivest. Introduction to Algorithms. MIT Press, 1990.

[11] G. H. Golub and C. F. V. Loan. Matrix Computations. Johns Hopkins, 1989. A. Geometric Algorithm Combinatorial

Optimization. Algorithms and Combinatorics 2.
SpringerVerlag, 1994. Second corrected edition.

[13] Y.-T. He, I. Al-Azzoni, and D. Down. MARO-MinDrift affinity routing for resource management in heterogeneous computing systems. In CASCON '07: Proceedings of the 2007 Conference of the Center for Advanced Studies on Collaborative Research, pages 71-85, New York, 2007. ACM.

[14] B. Hong and V. Prasanna. Distributed adaptive task allocation in heterogeneous computing environments to maximize throughput. In International Parallel and Distributed Processing Symposium IPDPS'2004. IEEE Computer Society Press, 2004.

[15] R. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher, editors, Complexity of Computer Computations (Proc. Sympos., IBM Thomas J. Watson Res. Center, Yorktown Heights, NY, 1972), pages 85-103. Plenum, NY, 1972.

[16] L. G. Khachiyan. A polynomial algorithm in linear programming. Doklady Akademii Nauk SSSR, 224:1093-1096, 1979. English Translation: Soviet Mathematics Doklady, Volume 20, pp. 191-194.

[17] A. Legrand, L. Marchal, and Y. Robert. Optimizing the steady-state throughput of scatter and reduce operations on heterogeneous platforms. Journal of Parallel and Distributed Computing, 65(12):1497-1514, 2005.

[18] L. Marchal. Communications collectives et ordonnancement en régime permanent sur plates-formes hétérogènes. Thèse de doctorat, École Normale Supérieure de Lyon, France, Oct. 2006.

[19] A. Schrijver. Combinatorial Optimization: Polyhedra and Efficiency, volume 24 of Algorithms and Combinatorics. Springer-Verlag, 2003.

8 Chapter 8: Divisible Load Scheduling

- [1] M. Adler, Y. Gong, and A. Rosenberg. On “exploiting” nodeheterogeneous clusters optimally. *Theory of Computing Systems*, 42(4):465–487, 2008.
- [2] D. T. Altılar and Y. Paker. An optimal scheduling algorithm for parallel video processing. In *International Conference on Multimedia Computing and Systems*, pages 245–248, July 1998.
- [3] D. T. Altılar and Y. Parker. Optimal scheduling algorithms for communication constrained parallel processing. In *Euro-Par 2002 Parallel Processing*, volume 2400 of *LNCS*, pages 197–206, 2002.
- [4] O. Beaumont, H. Casanova, A. Legrand, Y. Robert, and Y. Yang. Scheduling divisible loads on star and tree networks: Results and open problems. Research report 4916, INRIA, 2003.
- [5] O. Beaumont, H. Casanova, A. Legrand, Y. Robert, and Y. Yang. Scheduling divisible loads on star and tree networks: Results and open problems. *IEEE Transactions on Parallel and Distributed Systems*, 16(3):207–218, Mar. 2005.
- [6] O. Beaumont, L. Marchal, and Y. Robert. Scheduling divisible loads with return messages on heterogeneous master-worker platforms. In *International Conference on High Performance Computing HiPC’2005*, pages 123–132. Springer-Verlag, 2005.
- [7] D. P. Bertsekas. *Constrained optimization and Lagrange Multiplier methods*. Athena Scientific, 1996.
- [8] V. Bharadwaj, D. Ghose, and V. Mani. Multi-installment load distribution in tree networks with delays. *IEEE Transactions on Aerospace and Electronic Systems*, 31(2):555–567, Apr. 1995.
- [9] V. Bharadwaj, D. Ghose, V. Mani, and T. G. Robertazzi. *Scheduling Divisible Loads in Parallel and Distributed Systems*. IEEE Computer Society Press, 1996.
- [10] J. Błazewicz, M. Drozdowski, and M. Markiewicz. Divisible task scheduling concept and verification. *Parallel Computing*, 25(1):87–98, Jan. 1999.
- [11] M. Drozdowski. *Selected Problems of Scheduling Tasks in Multiprocessor Computer Systems*. Ph.D. thesis, Instytut

Informatyki Politechnika Poznanska, Poznan, 1997. scatter for grid computing. *Parallel Computing*, 30(8):923-946, 2004.

[13] R. Graham, D. Knuth, and O. Patashnik. *Concrete Mathematics: A Foundation for Computer Science*. Wesley, 1994.

[14] C.-K. Lee and M. Hamdi. Parallel image processing applications on a network of workstations. *Parallel Computing*, 21(1):137-160, 1995.

[15] T. G. Robertazzi. Ten reasons to use divisible load theory. *Computer*, 36(5):63-68, May 2003.

[16] T. Saif and M. Parashar. Understanding the behavior and performance of non-blocking communications in MPI. In *Euro-Par 2004 Parallel Processing*, volume 3149 of LNCS, pages 173-182. Springer, Dec. 2004.

[17] A. Schrijver. *Theory of Linear and Integer Programming*. John Wiley & Sons, New York, 1986.

[18] Y. Yang and H. Casanova. Extensions to the multi-installment algorithm: Affine cost and output data transfers. Technical Report CS20030754, Dept. of Computer Science and Eng., Univ. of California, San Diego, July 2003.

[19] Y. Yang, H. Casanova, M. Drozdowski, M. Lawenda, and A. Legrand. On the complexity of multi-round divisible load scheduling. Research Report 6096, INRIA, Jan. 2007.

[20] Y. Yang, K. van der Raadt, and H. Casanova. Multiround algorithms for scheduling divisible loads. *IEEE Transactions on Parallel and Distributed Systems*, 16(11):1092-1102, 2005.

9 Chapter 9: Multi-Objective Scheduling

- [1] A. Agnetis, P. Mirchandani, D. Pacciarelli, and A. Pacifici. Scheduling problems with two competing agents. *Operations Research*, 52(2):229– 242, 2004.
- [2] S. Albers and H. Fujiwara. Energy-efficient algorithms for flow time minimization. In *23rd International Symposium on Theoretical Aspects of Computer Science (STACS)*, volume 3884 of LNCS, pages 621–633. Springer, 2006.
- [3] E. Angel, E. Bampis, and A. Kononov. A FPTAS for approximating the Pareto curve of the unrelated parallel machines scheduling problem with costs. In *Algorithms ESA 2001*, volume 2161 of LNCS, pages 194–205. Springer, 2001.
- [4] J. Aslam, A. Rasala, C. Stein, and N. Young. Improved bicriteria existence theorems for scheduling. In *SODA '99: Proceedings of the Tenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 846–847, 1999.
- [5] I. Assayad, A. Girault, and H. Kalla. A bi-criteria scheduling heuristic for distributed embedded systems under reliability and real-time constraints. In *2004 International Conference on Dependable Systems and Networks (DSN'04)*, pages 347–356, June 2004.
- [6] K. Baker and J. Smith. A multiple-criterion model for machine scheduling. *Journal of Scheduling*, 6:7–16, 2003.
- [7] M. Bender, S. Muthukrishnan, and R. Rajaraman. Improved algorithms 13th Annual Symposium on Discrete Algorithms (SODA), pages 762–771, 2002.
- [8] D. Bertsekas and R. Gallager. *Data Networks*. Prentice-Hall, 1987.
- [9] P. Brucker. *Scheduling Algorithms*. Springer-Verlag, Berlin, 5th edition, 2007.
- [10] D. Bunde. Power-aware scheduling for makespan and flow. In *18th Annual ACM Symposium on Parallel Algorithms and Architectures*, pages 190–196, 2006.
- [11] Y. Caniou and E. Jeannot. Experimental study of multi-criteria scheduling heuristics for GridRPC systems. In *Euro-Par 2004 Parallel Processing*, volume 3149 of LNCS, pages 1048–1055. Springer, 2004.
- [12] C. Chekuri, R. Motwani, B. Natarajan, and C. Stein.

Approximation techniques for average completion time scheduling. In SODA '97: Proceedings of the Eighth Annual ACM-SIAM Symposium on Discrete Algorithms, pages 609-618, 1997.

[13] M. Chen. Individual monotonicity and the leximin solution. *Economic Theory*, 15(2):353-365, 2000.

[14] M. Detyniecki. Mathematical aggregation operators and their application to video querying. PhD thesis, Universite' Paris VI, 2000. <http://www.lip6.fr/reports/lip6.2001.002.html>.

[15] J. Dongarra, E. Jeannot, E. Saule, and Z. Shi. Bi-objective scheduling algorithms for optimizing makespan and reliability on heterogeneous systems. In SPAA '07: Proceedings of the Nineteenth Annual ACM Symposium on Parallelism in Algorithms and Architectures, pages 280-288. ACM Press, June 2007.

[16] P.-F. Dutot, L. Eyraud, G. Mounie', and D. Trystram. Bi-criteria algorithm for scheduling jobs on cluster platforms. In Symposium on Parallel Algorithms and Architectures, pages 125-132. ACM Press, 2004.

[17] D. Feitelson. Workload modeling for computer systems performance evaluation. <http://www.cs.huji.ac.il/~feit/wlmod/>.

[18] M. R. Garey and D. S. Johnson. Computers and Intractability. A Guide to the Theory of NP-Completeness. W. H. Freeman & Co, San Francisco, 1979.

[19] R. Graybill and R. Melhem, editors. Power Aware Computing. Series in Computer Science. Kluwer Academic/Plenum Publishers, New York, May 2002.

[20] K. Ho. Dual criteria optimization problems for imprecise computation tasks. In Leung [25], chapter 35. *Journal of tional Research*, 167(3):592-623, Dec. 2004.

[22] E. Jeannot, E. Saule, and D. Trystram. Bi-objective approximation scheme for makespan and reliability optimization on uniform parallel machines. In Euro-Par 2008, volume 5168 of LNCS. Springer, Aug. 2008.

[23] M. Kostreva, W. Ogryczak, and A. Wierzbicki. Equitable aggregations and multiple criteria analysis. *European Journal of Operational Research*, 158:362-377, 2004.

[24] C. Laforest. A tricriteria approximation algorithm for Steiner tree in graphs. Technical Report 69-2001, LaMI, 2001.

[25] J. Y.-T. Leung, editor. Handbook of Scheduling: Algorithms, Models, and Performance Analysis. Chapman & Hall/CRC, 2004.

[26] A. Marshall and I. Olkin. Inequalities: Theory of Majorization and Its Applications. Academic Press, 1979.

[27] J. Mo and J. Warland. Fair end-to-end window-based congestion control. In Proc. of SPIE '98: International Symposium on Voice, Video and Data Communications, 1998.

[28] C. Papadimitriou and M. Yannakakis. On the approximability of tradeoffs and optimal access of web sources. In 41st Annual Symposium on Foundations of Computer Science, pages 86-92, 2000.

[29] B. Radunovic and J. Le Boudec. A Unified Framework for Max-Min and Min-Max Fairness with Applications. IEEE/ACM Transactions on Networking, 15(5):1073-1083, Oct. 2007.

[30] A. Rasala, C. Stein, E. Torng, and P. Uthaisombut. Existence theorems, lower bounds and algorithms for scheduling to meet two objectives. In SODA '02: Proceedings of the Thirteenth Annual ACM-SIAM Symposium on Discrete Algorithms, pages 723-731, 2002.

[31] J. Rawls. The Theory of Justice. Harvard Univ. Press, 1971.

[32] P. Samuelson and W. Nordhaus. Economics. McGraw-Hill, 16th edition, 1998.

[33] E. Saule, P.-F. Dutot, and G. Mounie'. Scheduling with storage constraints. In IEEE International Symposium on Parallel and Distributed Processing, Apr. 2008.

[34] S. Shatz and J. Wang. Task allocation for maximizing reliability of distributed computer systems. IEEE Transactions on Computer, 41(9):1156- 1169, Sept. 1992.
machines with In Proceedings of the Fourth Annual ACM/SIGACT-SIAM Symposium on Discrete Algorithms, pages 448-454, 1993.

[36] D. B. Shmoys, J. Wein, and D. P. Williamson. Scheduling parallel machines on-line. SIAM Journal on Computing, 24(6):1313-1331, Dec. 1995.

- [37] M. Skutella. Approximation algorithms for the discrete time-cost tradeoff problem. *Mathematics of Operations Research*, 23(4):909-929, 1998.
- [38] S. Srinivasan and N. Jha. Safety and reliability driven task allocation in distributed systems. *IEEE Transactions on Parallel and Distributed Systems*, 10(3):238-251, Mar. 1999.
- [39] C. Stein and J. Wein. On the existence of schedules that are near-optimal for both makespan and total weighted completion time. *Operational Research Letters*, 21(3):115-122, Oct. 1997.
- [40] R. Steueur. Multiple Criteria optimization: theory, computation and application. John Wiley, 1986.
- [41] W. Stromquist. How to cut a cake fairly. *The American Mathematical Monthly*, 87(8):640-644, 1980.
- [42] V. T'kindt and J. Billaut. *Multicriteria Scheduling*. Springer, 2007.
- [43] V. T'kindt, K. Bouibede-Hocine, and C. Esswein. Counting and enumeration complexity with application to multicriteria scheduling. *Annals of Operations Research*, Apr. 2007.
- [44] M. Voorneveld. Characterization of Pareto dominance. *Operations Research Letters*, 31:7-11, 2003.

10 Chapter 10: Comparisons of Stochastic Task-Resource Systems

- [1] E. Altman, B. Gaujal, and A. Hordijk. Optimal open-loop control of vacations, polling and service assignment. *Queueing Systems*, 36:303– 325, 2000.
- [2] E. Altman, B. Gaujal, and A. Hordijk. Discrete-Event Control of Stochastic Networks: Multimodularity and Regularity, volume 1829 of LNM. Springer-Verlag, 2003.
- [3] F. Baccelli and P. Brémaud. Elements of Queueing Theory. SpringerVerlag, 1994.
- [4] V. Berten, A. Busic, B. Gaujal, and J.-M. Vincent. Can we use perfect simulation for non-monotonic markovian systems? In *Roadef*, pages 166–167, Clermont-Ferrand, 2008.
- [5] V. Berten and B. Gaujal. Brokering strategies in computational grids using stochastic prediction models. *Parallel Computing*, 33(4-5):238– 249, 2007. Special Issue on Large Scale Grids.
- [6] A. Bouillard and B. Gaujal. Backward coupling in bounded free-choice nets under markovian and non-markovian assumptions. *Journal of Discrete Event Dynamics Systems: Theory and Applications*, 18(4):473– 498, 2008. Special issue of selected papers from the Valuetools conference.
- [7] B. Dodin. Bounding the project completion time distributions networks. *Operations Research*, 33(4):862–881, 1985.
- [8] J. Dopper, B. Gaujal, and J.-M. Vincent. Bounds for the coupling time in queueing networks perfect simulation. In *Numerical Solutions for June 2006. ebration of the 100th anniversary of Markov*.
- [9] J.-M. Fourneau, I. Y. Kadi, N. Pekergin, J. Vienne, and J.-M. Vincent. Perfect Simulation and Monotone Stochastic Bounds. In *2nd International Conference Valuetools’07*, Nantes, France, Oct. 2007.
- [10] B. Gaujal, A. Hordijk, and D. V. der Laan. On the optimal open-loop control policy for deterministic and exponential polling systems. *Probability in Engineering and Informational Sciences*, 21:157–187, 2007.
- [11] J. Kamburowski. Bounding the distribution of project duration in PERT networks. *Operation Research Letters*,

12:17-22, 1992.

[12] S. Karlin. Dynamic inventory policy with varying stochastic demands. *Management Sciences*, 6:231-258, 1960.

[13] G. Koole. Structural results for the control of queueing systems using event-based dynamic programming. Technical Report WS-461, Vrije Universiteit Amsterdam, 1996.

[14] A. Müller and D. Stoyan. Comparison methods for stochastic models and risks. *Wiley Series in Probability and Statistics*. Wiley, 2002.

[15] J. G. Propp and D. B. Wilson. Exact sampling with coupled Markov chains and applications to statistical mechanics. *Random Structures and Algorithms*, 9(1-2):223-252, 1996.

[16] S. M. Ross. Average delay in queues with non-stationary Poisson arrivals. *Journal of Applied Probability*, 15:602-609, 1978.

[17] S. M. Ross. *Introduction to Probability Models*. Academic Press, 2003.

[18] J. Shanthikumar and D. D. Yao. Bivariate characterization of some stochastic order relations. *Advances in Applied Probability*, 23(3):642- 659, 1991.

[19] V. Strassen. The existence of probability measures with given marginals. *Annals of Mathematical Statistics*, 36:423-439, 1965.

[20] J.-M. Vincent. Perfect Simulation of Monotone Systems for Rare Event Probability Estimation. In *Winter Simulation Conference*, Orlando, Dec. 2005.

[21] J.-M. Vincent and J. Vienne. Perfect simulation of index based routing queueing networks. *Performance Evaluation Review*, 34(2):24-25, 2006.

[22] N. Yazici-Pekergin and J.-M. Vincent. Stochastic bounds on execution times of parallel programs. *IEEE Trans. Software Eng.*, 17(10):1005- 1012, 1991.

11 Chapter 11: The Influence of Platform Models on Scheduling Techniques

- [1] A. Alexandrov, M. Ionescu, K. Schauser, and C. Scheiman. LogGP: Incorporating long messages into the LogP model for parallel computation. *Journal of Parallel and Distributed Computing*, 44(1):71-79, 1997.
- [2] M. Banikazemi, V. Moorthy, and D. K. Panda. Efficient collective communication on heterogeneous networks of workstations. In *Proceedings of the 27th International Conference on Parallel Processing (ICPP'98)*. IEEE Computer Society Press, 1998.
- [3] M. Banikazemi, J. Sampathkumar, S. Prabhu, D. Panda, and P. Sadayappan. Communication modeling of heterogeneous networks of workstations for performance characterization of collective operations. In *HCW'99, the 8th Heterogeneous Computing Workshop*, pages 125-133. IEEE Computer Society Press, 1999.
- [4] A. Bar-Noy, S. Guha, J. S. Naor, and B. Schieber. Message multicasting in heterogeneous networks. *SIAM Journal on Computing*, 30(2):347- 358, 2000.
- [5] O. Beaumont, L. Eyraud-Dubois, and N. Bonichon. Scheduling divisible workload on heterogeneous platforms under bounded multi-port model. In *Heterogeneous Computing Workshop HCW'2008. Proceedings of IPDPS 2008*, IEEE Computer Society Press, 2008.
- [6] P. Bhat, C. Raghavendra, and V. Prasanna. Efficient collective communication in distributed heterogeneous systems. In *ICDCS'99 19th Systems*, pages 24. IEEE Computer Society Press, 1999.
- [7] P. Bhat, C. Raghavendra, and V. Prasanna. Efficient collective communication in distributed heterogeneous systems. *Journal of Parallel and Distributed Computing*, 63:251-263, 2003.
- [8] J. Cohen, E. Jeannot, N. Padoy, and F. Wagner. Message Scheduling for Parallel Data Redistribution between Clusters. *IEEE Transactions on Parallel and Distributed Systems*, 17:1163-1175, 2006.
- [9] P. Crescenzi, D. Xiaotie, and C. Papadimitriou. On approximating a scheduling problem. *Combinatorial Optimization*, 5:287-297, 2001.

[10] D. Culler, R. Karp, D. Patterson, A. Sahay, E. Santos, K. Schauer, R. Subramonian, and T. von Eicken. LogP: a practical model of parallel computation. *Communication of the ACM*, 39(11):78-85, 1996.

[11] S. Even, A. Itai, and A. Shamir. On the complexity of timetable and multicommodity flow problem. *SIAM Journal on Computing*, 5:691-703, 1976.

[12] I. Gopal and C. Wong. Minimizing the number of switching in an SS/TDMA system. *IEEE Transactions on Communications*, 33(6):497- 501, 1985.

[13] R. W. Hockney. The communication challenge for MPP: Intel Paragon and Meiko CS-2. *Parallel Computing*, 20:389-398, 1994.

[14] B. Hong and V. Prasanna. Distributed adaptive task allocation in heterogeneous computing environments to maximize throughput. In *International Parallel and Distributed Processing Symposium IPDPS'2004*. IEEE Computer Society Press, 2004.

[15] S. Khuller and Y. A. Kim. On broadcasting in heterogenous networks. In *Proceedings of the Fifteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1011-1020. Society for Industrial and Applied Mathematics, 2004.

[16] T. Kielmann, H. E. Bal, and K. Verstoep. Fast measurement of LogP parameters for message passing platforms. In *Parallel and Distributed Processing: 15 IPDPS 2000 Workshops*, volume 1800 of LNCS. Springer, 2000.

[17] B. Kreaseck, L. Carter, H. Casanova, and J. Ferrante. On the interference of communication on computation in java. In *IPDPS*, 2004.

[18] A. Legrand, H. Renard, Y. Robert, and F. Vivien. Mapping and loadbalancing iterative computations on heterogeneous clusters with shared links. *IEEE Transactions on Parallel Distributed Systems*, 15(6):546- 558, 2004.
heterogeneous cluster tems. *Journal of Algorithms*, 42(1):135-152, 2002.

[20] S. H. Low. A duality model of TCP and queue management algorithms. *IEEE/ACM Transactions on Networking*, 2003.

[21] L. Marchal, Y. Yang, H. Casanova, and Y. Robert. Steady-state scheduling of multiple divisible load

applications on wide-area distributed computing platforms.
International Journal of High Performance Computing
Applications, 20(3):365-381, 2006.

[22] L. Massoulié and J. Roberts. Bandwidth sharing:
Objectives and algorithms. In INFOCOM (3), pages 1395-1403,
1999.

[23] T. Saif and M. Parashar. Understanding the behavior
and performance of non-blocking communications in MPI. In
Proceedings of Euro-Par 2004: Parallel Processing, LNCS
3149, pages 173-182. Springer, 2004.

[24] Y. Yang, H. Casanova, M. Drozdowski, M. Lawenda, and
A. Legrand. On the complexity of multi-round divisible load
scheduling. Research Report 6096, INRIA, Jan. 2007.

[25] Y. Yang, K. van der Raadt, and H. Casanova. Multiround
algorithms for scheduling divisible loads. IEEE
Transactions on Parallel and Distributed Systems,
16(11):1092-1102, 2005.